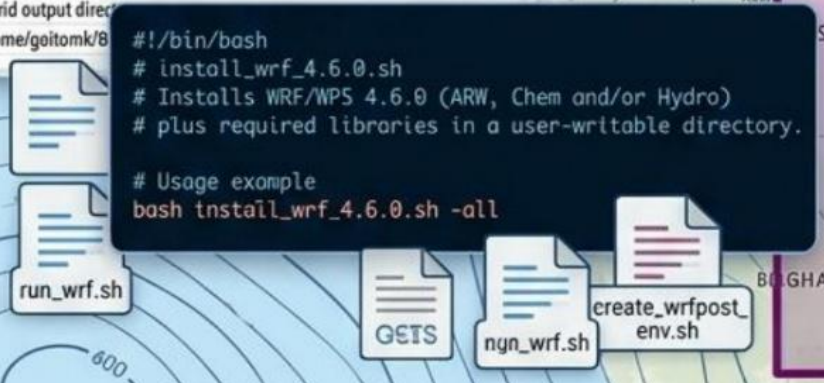
Automated WRF Installation and Domain Configuration

A Complete Guide to Automated WRF-ARW, WRF-Chem and WRF-Hydro Installation, Domain Configuration, and Post-processing.





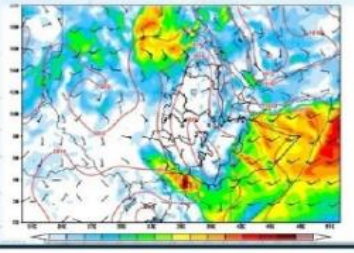


Goitom Kelem

October 2026

`install_wrf_4.6.0.sh`
`WRF-domain-Wizard.html`
`run_wrf.sh`
`create_wrfpost_env.sh`

WRF 4.6.0
ARW | Chem | Hydro



Contents

1. Scope & Objectives 1
Core & Extended Frameworks: 1
1.1. Repository Structure & Key Components 1
Cloning the repository 1
Supported build types 2
1.3. Prerequisites 2
Quick Start 3
1.4. Command-Line Options 3
Notes on behavior: 3
1.5. Directory Layout 4
Installation layout 4
1. 6. Execution Flow 5
Helper functions 5
1.7. Libraries 5
1.8. WRF Build Procedure 6
1.9. WPS Build Procedure 6
1.10. Geography Data 7
1.11. Environment Setup 7
1.12. Verifying the Installation 7
1.13. Typical Run Hints 8
1.14. Re-running, Resuming and Rebuilding 8
1.15. Troubleshooting 8
1.16. Known Limitations and Review Notes 9
1.17. Uninstalling 9
1.18. Quick Reference 10
1.19. Setting up domain directory and link WSP/WRF files 10
Chapter two 14
2. Configure WRF domain 14
2.1 WRF Domain Wizard: User Guide 14
Before you start 15
Step 1. Draw the outer domain (d01) 16
Step 2. Add nested domains (d02, d03, ...) 17

Step 3. Select, move and resize boxes 17
Step 4. Choose the projection 18
Step 5. Set the timing and run options 19
Step 6. Set the data and output directories 19
Step 7. Generate and export the namelists 20
Checking before a real run 20
Troubleshooting 21
Chapter three 23
WRF Modelling System and Preprocessing System 23
3.0 The WRF Modeling System Program Components 23
3.1 WRF Preprocessing System (WPS) 23
3.2 Spatial simulation domain for East Africa 24
3.3 Input gridded time-varying data for model run 24
3.3.1 Static Geographical Data: 25
Learning Outcomes 25
3.4 WRF Preprocessing System 26
3.4.1 WRF Preprocessing System 26
3.5 Prepare input data for the WRF model 27
2. Prerequisites 29
5. Step-by-Step Technical Execution Workflow 30
6. Integration with WRF Preprocessing System (WPS) 31
Step 1: Navigate to the Domain Directory 31
Step 2: Run Geogrid 31
Step 3: Link GRIB Files 31
Step 4: Link the Variable Table 32
Step 5: Run Ungrib 32
Step 6: Run Metgrid 32
3.4.2 WRF Model 32
Step 1: Run real 36
Step 2: Run WRF 39
Chapter four 45
4. WRF Post-Processing Using Python 45
4.1 Major applications of Python in WRF post-processing 45

4.1.1 Recommended Python libraries 46
4.2 Install the main packages 47
Appendix B. References and assumptions 51

Overview

This document provides an overview of the user-space installation framework for WRF/WPS version 4.6.0, supporting ARW, Chem, and Hydro configurations along with all required dependency libraries.

All automation scripts, configuration templates, pre-installation checks, and post-installation validation tools are maintained in the following GitHub repository:

👉 https://github.com/goikelem/WRF-installation-automation.git

1. Scope & Objectives

User-Space Deployment: Installs all compilers, libraries, and model binaries entirely within a user-defined directory (no sudo or root privileges required). And Domain configuration. Comprehensive Stack: Automatically builds the complete dependency chain including serial/parallel `NetCDF, HDF4, HDF5, Zlib, Libpng, Jasper, MPICH`, and optional geo-processing/geogrid libraries.

Core & Extended Frameworks:

WRF/WPS ARW: Standard Advanced Research WRF dynamical core and pre-processing system. WRF-Chem: Integrated chemistry-meteorology modeling framework. WRF-Hydro: Hydrological extension coupled with WRF.

1.1. Repository Structure & Key Components

Cloning the repository

```
git clone [https://github.com/goikelem/WRF-installation-
automation.git](https://github.com/goikelem/WRF-installation-
automation.git)
```

provides access to a structured set of scripts and documentation: README.md: Step-by-step master guide detailing system requirements, environment variable setups, compiler selections (GNU/Intel), and troubleshooting tips.

install_wrf_4.6.0.sh builds a complete WRF modelling stack from source without root privileges. In a single run it:

1. Downloads and compiles nine supporting libraries into a private prefix.
2. Downloads, configures and compiles WRF 4.6.0 and WPS 4.6.0 for each requested build type.

3. Downloads WPS geographical data (and extra datasets for Chem).
4. Generates environment files (an Environment Modules modulefile plus Bash and Tcsh source files) and a human-readable "Do List" with loading instructions.

The script combines two inspirations: download links and build steps come from install_wrf_and_libraries.sh, while the environment handling (templated files with placeholders, modulefiles instead of editing ~/.bashrc, a Do List, and --homedir/--continue/--force/--proxy flags).

Supported build types

Flag	Build type	What it adds
`-arw`	ARW	Standard WRF-ARW dynamical core, em_real case
`-chem`	Chem	WRF-Chem with KPP chemistry (WRF_CHEM=1, WRF_KPP=1), Flex/YACC, extra chemistry geography data
`-hydro`	Hydro	WRF-Hydro (v5.3.0 NDHMS code copied into hydro/, WRF_HYDRO=1)
`-all`	All three	Equivalent to -arw -chem -hydro

Each type gets its own WRF and WPS source tree, so the three configurations never overwrite one another. All types share the same libraries and the same WPS_GEOG directory.

1.3. Prerequisites

The script does not install system packages. The following must already exist on the machine:

Or, if you have admin privileges, install the system packages for Debian and Ubuntu using the first script and for redhat family using the second one.

```
install_wrf_system_dependencies_debian_ubuntu_packages.sh
install_wrf_system_dependencies_redhatfamily_packages.sh
```

- Compilers: gcc, g++, gfortran (the script hard-codes these as CC, CXX, FC, F77).
- Build tools: make, tar, sed, awk, perl, csh/tcsh (WRF's configure and compile scripts need csh), m4.
- Downloaders: wget or curl.
- Network access to GitHub, the HDF Group, mpich.org, SourceForge, invisible-island.net and www2.mmm.ucar.edu.

- Disk space: plan for roughly 30 GB free (libraries, three WRF/WPS trees, tarballs, and the mandatory high-resolution geography data). Chem geography data adds more.
- Time: libraries take about 20-40 minutes in total; each WRF compile typically takes 20-40+ minutes.

Optional: the `modulecmd`/`module` command (Environment Modules). Without it the script falls back to instructing you to source the generated shell files.

Quick Start

```
bash install_wrf_4.6.0.sh -arw
```

Build everything into a custom location:

```
bash install_wrf_4.6.0.sh --homedir /scratch/me/wrf -all
```

After it finishes, read `$WRF_HOME/WRF-DoList.txt` and load an environment:

```
source $WRF_HOME/Build_WRF/etc/modulefiles/WRFARW.sh
```

1.4. Command-Line Options

Option	Argument	Default	Effect
`-arw, -chem, -hydro`	none	none	Add that build type. At least one is required.
`-all`	none	none	Replace the type list with ARW, Chem and Hydro.
`--homedir`	DIR	$HOME/Build_WRF	Top-level install directory (created if missing).
`--proxy`	HOST:PORT	proxies cleared	Sets http_proxy and https_proxy.
`--continue`	none	off	Skip libraries that have a done-marker; tarballs are kept.
--force	none	off	Re-download, re-unpack and rebuild everything.
--nogeog	none	off	Skip the WPS_GEOG download.
-h, --help	none	none	Print the usage block from the script header.

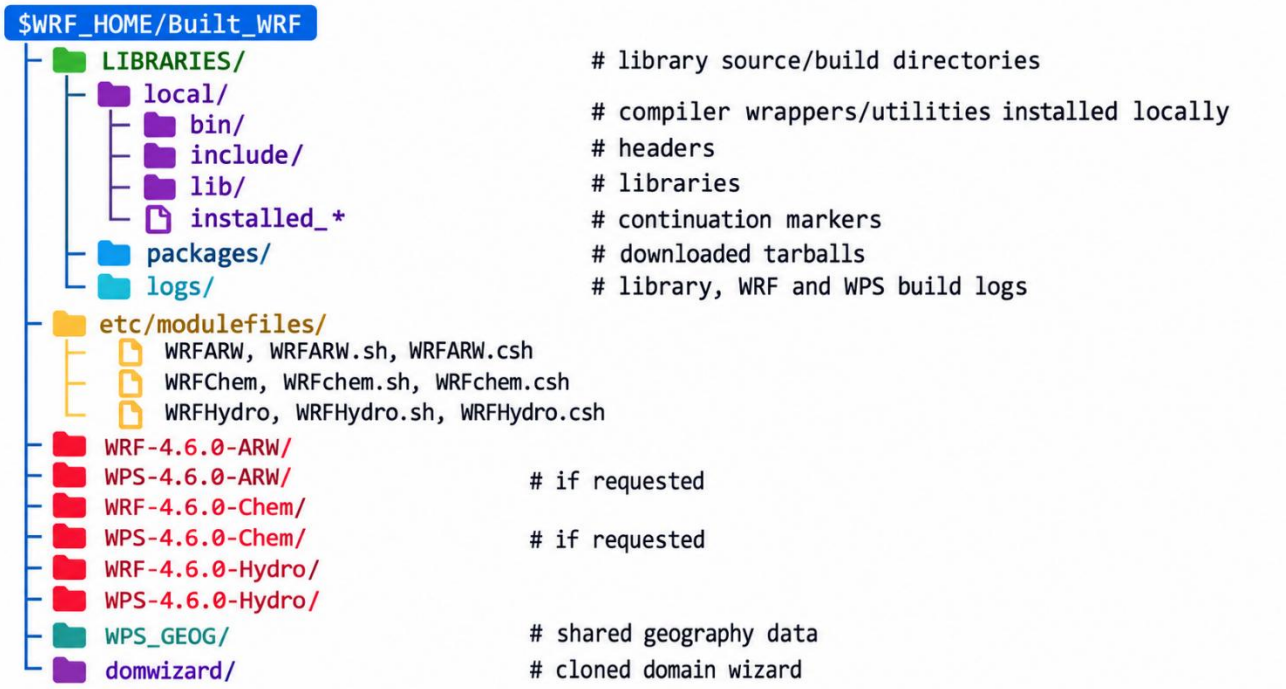
Notes on behavior:

- Flags can be combined and repeated; duplicate build types are removed with `awk '!s[$0]++'`.
- With no build type, the script prints the three example commands and exits with status 1.
- Unrecognized options print an error plus usage and exit 1.

1.5. Directory Layout

All paths are relative to `WRF_HOME` (default `$WRF_HOME/Build_WRF`).

Installation layout



The exact tree depends on selected build types and whether each stage completes. Keep packages/ and logs/ until the installation has been validated.

Path	Contents
`LIBRARIES/`	Unpacked library source trees (build area)
`local/`	Install prefix (bin, lib, include) shared by all builds; also holds .installed_<lib> markers
`packages/`	Saved tarballs, so re-runs do not re-download
`etc/modulefiles/`	Generated WRF<Type> modulefile, WRF<Type>.sh, WRF<Type>.csh
`logs/`	lib_<name>.log, wrf_<Type>.log, wps_<Type>.log
`WRF-4.6.0-<Type>/`	Compiled WRF tree for each type
`WPS-4.6.0-<Type>/`	Compiled WPS tree for each type

Path	Contents
WPS_GEOG/	Shared static geography data
$HOME/WRF-DoList.txt	Instructions for loading the environment

1. 6. Execution Flow

1. Initialization. Sets set -eo pipefail, exports LC_ALL=C and unsets http_proxy/https_proxy (so a stale shell proxy cannot break downloads). --proxy re-enables them.
2. Argument parsing. Fills the TYPES array and option flags.
3. Path setup. Resolves WRF_HOME to an absolute path, creates the directory tree, detects NPROCS via nproc (fallback 4), and caps make -j at 8 (MAKEJ).
4. Library build. Loops over nine libraries (Section 7), writing each log to logs/.
5. Per-type loop. For each build type: build_wrf, build_wps, write_environment_files.
6. Geography data. setup_geog downloads WPS_GEOG once (and Chem extras if Chem was requested).
7. Do List. Writes $HOME/WRF-DoList.txt and prints it together with a summary of executables.

Helper functions

Function	Purpose
fetch URL FILE	Downloads to packages/. Skips if a non-empty file exists (unless --force). Tries wget first and falls back to curl -fL. Aborts if the result is empty.
unpack TAR DIR	Extracts into LIBRARIES/ unless the directory already exists (unless --force).
is_done, mark_done	Marker files local/.installed_<lib>. Markers are honored only when --continue is given and --force is not.
load_build_env TYPE	Exports NETCDF, HDF5, JASPERLIB/INC, PATH, LD_LIBRARY_PATH and the type-specific variables for the current script session. It first unsets WRF_CHEM, WRF_KPP, WRF_HYDRO, FLEX_LIB_DIR, YACC so builds do not contaminate one another.

1.7. Libraries

All libraries are compiled with -O3 -fPIC using GNU compilers and installed into local/.

No	Library	Version	Notes
1	zlib	1.2.13	Required by HDF5
2	HDF5	1.12.2	Serial (--enable-parallel=no), with --enable-hl --enable-fortran
3	NetCDF-C	4.9.2	--disable-dap --disable-byterange (removes the libcurl dependency)
4	NetCDF-Fortran	4.6.1	Links -lnetcdf -lhdf5_hl -lhdf5 -lz
5	MPICH	4.1.2	Provides mpif90/mpicc for the dmpar build
6	libpng	1.6.39	Needed by WPS GRIB2 output
7	JasPer	1.900.1	JPEG2000 support for WPS GRIB2
8	Flex	2.6.4	Needed by WRF-Chem KPP
9	byacc	latest	Needed by WRF-Chem KPP

Flex and byacc are only required by Chem but are always built because they are inexpensive.

If any library fails, the script prints the log path and the last 20 lines of the log, then exits.

1.8. WRF Build Procedure

build_wrf TYPE performs these steps:

1. Skip check. If WRF-4.6.0-<Type>/main/wrf.exe exists and --force is not set, the type is skipped.
2. Environment. Calls load_build_env.
3. Source. Downloads the GitHub release tarball and extracts it. Because the tarball directory name can be WRFV4.6.0 or WRF-4.6.0, the script renames to WRFV4.6.0 when needed.
4. Hydro only. Downloads wrf_hydro_nwm_public v5.3.0 and copies trunk/NDHMS/* into WRFV4.6.0/hydro/.
5. Configure. Pipes 34 (GNU gfortran/gcc, dmpar) and 1 (basic nesting) into ./configure.
6. Compatibility patch. For gfortran major versions 6 and 7, removes -DBUILD_RRTMG_FAST=1 from configure.wrf.
7. Compile. Runs ./compile em_real. Failure is detected by the string *"Problems building executables"* in the log or by a missing main/wrf.exe.
8. Finalize. Renames WRFV4.6.0 to WRF-4.6.0-<Type>.

A WRF compile typically produces wrf.exe, real.exe, ndown.exe and tc.exe in main/.

1.9. WPS Build Procedure

build_wps TYPE downloads WPS 4.6.0, renames it to WPS-4.6.0-<Type>, runs ./clean, exports WRF_DIR pointing at the matching WRF tree, then configures with option 3 (Linux gfortran,

dmpar) and compiles. Success requires geogrid.exe, ungrib.exe and metgrid.exe. Finally namelist.wps is edited so geog_data_path points at $WRF_HOME/WPS_GEOG/.

1.10. Geography Data

- The mandatory high-resolution dataset (geog_high_res_mandatory.tar.gz) is downloaded and extracted once into WPS_GEOG/.
- When Chem is among the requested types, these extra datasets are fetched if their directories are missing: modis_landuse_21class_30s, soiltype_top_2m, soiltype_bot_2m, albedo_ncep, maxsnowalb, erod, clayfrac_5m, sandfrac_5m.

`--nogeog` skips all of this; you must then provide WPS_GEOG yourself.

1.11. Environment Setup

write_environment_files writes three templates per build type and substitutes placeholders (@WRFHOME@, @PREFIX@, @WRFDIR@, @WPSDIR@, @NPROCS@, @WRFVER@, @TYPE@) with sed. Multi-line, type-specific blocks (@EXTRA_TCL@, @EXTRA_SH@, @EXTRA_CSH@) are substituted with awk.

File	Shell	Usage
WRF<Type>	Environment Modules (Tcl)	module load WRF<Type>
WRF<Type>.sh	Bash	source WRF<Type>.sh
WRF<Type>.csh	Tcsh	source WRF<Type>.csh

Variables defined include WRF_HOME, WRF_DIR, WPS_DIR, WPS_GEOG, NPROCS, DIR, PREFIX, the compiler set, NETCDF, HDF5, JASPERLIB, JASPERINC, FFLAGS, FCFLAGS, plus PATH and LD_LIBRARY_PATH prepends. Type-specific additions:

- Chem: WRF_CHEM=1, WRF_KPP=1, FLEX_LIB_DIR=$PREFIX/lib, YACC="yacc -d"
- Hydro: WRF_HYDRO=1
- ARW: none

Load only one build type per shell session. The Chem and Hydro flags would otherwise leak into an ARW workflow.

To use the modulefile, add this to ~/.bashrc:

```
export MODULEPATH="${MODULEPATH}:$WRF_HOME/Build_WRF/etc/modulefiles"
module load WRFARW
```

1.12. Verifying the Installation

```
source $WRF_HOME/Build_WRF/etc/modulefiles/WRFARW.sh
which mpif90 nc-config nf-config
```

```
nc-config --version
ls $WRF_DIR/main/*.exe
ls $WPS_DIR/*.exe
```

Expected: all executables exist and mpif90 resolves under `$PREFIX/bin`. For a functional check, run the em_real test case in `$WRF_HOME/WRF-4.6.0-ARW/test/em_real` with real input data, using `mpirun -np $NPROCS ./wrf.exe`.

1.13. Typical Run Hints

- Raise the stack limit before running: `ulimit -s unlimited`.
- Edit `namelist.wps` (domain, dates) then run `geogrid.exe`, `ungrib.exe`, `metgrid.exe` in `$WPS_DIR`.
- Link `met_em*` files into `$WRF_DIR/run`, edit `namelist.input`, then run `real.exe` and `wrf.exe` under `mpirun`.

1.14. Re-running, Resuming and Rebuilding

Situation	Command
Resume after a failed library build	`bash install_wrf_4.6.0.sh -arw --continue`
Add another build type later	`bash install_wrf_4.6.0.sh -chem --continue`
Full clean rebuild	`bash install_wrf_4.6.0.sh -arw --force`
Use a corporate proxy	`bash install_wrf_4.6.0.sh -arw --proxy proxy.example.org:3128`

Without `--continue`, libraries are rebuilt on every run even though tarballs are cached. WRF and WPS builds are skipped automatically if their executables already exist.

1.15. Troubleshooting

Symptom	Likely cause and action
ERROR: failed to download ...	URL moved or no network/proxy. Test the URL with curl -I, check --proxy, or place the tarball manually in packages/ under the expected name.
ERROR building <lib>	Open logs/lib_<lib>.log. Common causes are missing m4, missing compilers, or a stale LD_LIBRARY_PATH.
WRF configure shows no option 34	The menu numbering differs on your platform. Run ./configure manually once and adjust the echo 34 line.
Problems building executables	Read logs/wrf_<Type>.log and search for the first Error. Check the gfortran version and that NETCDF points to a library containing both C and Fortran parts.

Symptom	Likely cause and action
Chem build fails in KPP	Verify FLEX_LIB_DIR and YACC are set, and that flex/yacc exist in $PREFIX/bin.
`geogrid.exe` cannot open data	geog_data_path in namelist.wps is wrong or WPS_GEOG is incomplete.
`ungrib.exe` lacks GRIB2	JasPer/libpng were not found; check JASPERLIB/JASPERINC and rebuild WPS.
Runtime libnetcdf.so not found	Environment not loaded; source the environment file.

1.16. Known Limitations and Review Notes

Points worth knowing before relying on the script in production:

1. Proxy scheme. `--proxy` sets https_proxy to an https:// URL. Most proxies expect http:// for both variables; if HTTPS downloads fail behind a proxy, change the https_proxy assignment to use http://.
2. Hard-coded URLs. Several download locations (for example the HDF5 FTP-style path, SourceForge, JasPer from UCAR) can change or move. If a download fails, update the URL in the corresponding `build_*` function.
3. Do List loads only the first type. The Option 1 module instructions reference `WRF${TYPES[0]}`; the source-file section lists every type.
4. Library rebuild without `--continue`. Every run recompiles all libraries, which is slow on repeat invocations.
5. Single parallel configuration. Only dmpar (MPI) is built; there is no serial or OpenMP variant, and nesting is fixed to "basic".
6. Shared WPS_GEOG for Chem. Chem datasets are added into the common directory; ARW and Hydro simply ignore them.
7. Compiler assumptions. GNU toolchain only. Intel or other compilers require changing the CC/FC exports and the configure option numbers.
8. Failure detection is log-based. The compile steps use `|| true` followed by log/executable checks, so unusual failures that still leave an old executable in place could go unnoticed; `--force` avoids this.
9. Writes to $WRF_HOME. The Do List is always written to $WRF_HOME/WRF-DoList.txt regardless of `--homedir`.

1.17. Uninstalling

Everything lives under WRF_HOME, so removal is:

```
rm -rf $WRF_HOME/Build_WRF $HOME/WRF-DoList.txt
```

Then remove any MODULEPATH or module load lines you added to your login files.

1.18. Quick Reference

```
# install
bash install_wrf_4.6.0.sh -arw
bash install_wrf_4.6.0.sh -all --homedir /data/wrf --continue

# load
module load WRFARW                  # or: source
<homedir>/etc/modulefiles/WRFARW.sh

# logs
ls <homedir>/logs
```

1.19. Setting up domain directory and link WSP/WRF files

This bash script automates the setup of a customized working directory for running the Weather Research and Forecasting (***WRF***) model and its Preprocessing System (***WPS***). By pulling executables, configuration files, and custom automation scripts into a single folder, it allows you to run WRF for a specific domain independently of your main WRF build, preventing file clutter and configuration conflicts.
Environment Configuration: The script begins by exporting the core WRF environment variables (**WRF_HOME**, **WRF_WPS**, and **WRF_RUN**) to point to your specific **WPS** and **WRF** installation directories.

User Input & Directory Creation: It prompts the user to type a DOMAIN_NAME (e.g., "ethiowrf" or "goiwrf"). It then creates a main directory for this domain inside Build_WRF and exports a new variable, DOMAIN_DIR, to reference it.

Linking Executables: To avoid duplicating large binary files, the script creates symbolic links (ln -sf) for all ***.exe** files from the WPS directory (e.g., geogrid.exe, metgrid.exe, ungrib.exe). The link_grib.csh script is used for linking GRIB data.

All *.exe files from the WRF run directory (e.g., real.exe, wrf.exe, ndown.exe). Copying Namelists & Preprocessing Tables: Instead of linking, the script uses the copy (cp) command for critical configuration files:

`namelist.wps` and **`namelist.input`**: Copying these ensures that each domain folder has its own independent namelist. You can safely edit the simulation dates, grid sizes, and physics options for this specific domain without affecting your main WRF build or other domains.

geogrid, ungrib, and metgrid directories: Copying these folders ensures that the WPS executables have immediate access to necessary *.TBL and Vtable files, preventing common runtime errors like "Could not open GEOGRID.TBL".

Subdirectory Creation: It creates four organizational folders inside the domain directory to keep your workflow tidy:

wrf_output (for WRF model output)

GFS_Input (for downloaded GFS initial/boundary data)

ECMWF_Input (for ECMWF data)

Plots (for visualization outputs)

Fetching Custom Automation Scripts: Using curl, the script reaches out to your GitHub repositories and downloads two custom shell scripts directly into the domain folder:

download_025_gfs.sh (from your Automation-Automation-Utility-Scripts repo) and run_wrf.sh (from your Automation-Automation-Utility-Scripts repo). It then runs `chmod +x` on both files to ensure they are ready to be executed. Completion Message: Finally, it prints a success banner to the terminal, letting the user know the setup is complete and where the files are located.

```
#!/bin/bash

# Set WRF environment variables
export WRF_HOME="$HOME/Build_WRF"
export WRF_WPS="$HOME/Build_WRF/WPS-4.6.0-ARW"
export WRF_RUN="$HOME/Build_WRF/WRF-4.6.0-ARW/run"

# Prompt user for domain name from terminal
read -p "Enter the domain name: " DOMAIN_NAME

# Check if domain name is empty
if [ -z "$DOMAIN_NAME" ]; then
    echo "Error: Domain name cannot be empty."
    exit 1
fi

# Define the target directory path and export DOMAIN_DIR
export DOMAIN_DIR="$WRF_HOME/$DOMAIN_NAME"

# Create the target directory
mkdir -p "$DOMAIN_DIR"
echo "Created directory: $DOMAIN_DIR"

# Create links from WRF_WPS/*.exe to the created directory
ln -sf "$WRF_WPS"/*.exe "$DOMAIN_DIR"/

# Create link for WRF_WPS/link_grib.csh to the created directory
ln -sf "$WRF_WPS/link_grib.csh" "$DOMAIN_DIR"/

# Create links from WRF_RUN/*.exe to the created directory
ln -sf "$WRF_RUN"/*.exe "$DOMAIN_DIR"/

# Copy namelist.wps and namelist.input so they can be edited independently for
this domain
```

```
cp "$WRF_WPS/namelist.wps" "$DOMAIN_DIR"/
cp "$WRF_RUN/namelist.input" "$DOMAIN_DIR"/

# Copy required WPS directories for TBL and Vtable files (fixes GEOGRID.TBL
errors)
cp -r "$WRF_WPS/geogrid" "$DOMAIN_DIR"/
cp -r "$WRF_WPS/ungrib" "$DOMAIN_DIR"/
cp -r "$WRF_WPS/metgrid" "$DOMAIN_DIR"/

# Create required subdirectories inside the domain directory
mkdir -p "$DOMAIN_DIR"/wrf_output "$DOMAIN_DIR"/GFS_Input
"$DOMAIN_DIR"/ECMWF_Input "$DOMAIN_DIR"/Plots
echo "Created subdirectories: wrf_output, GFS_Input, ECMWF_Input, Plots"

# Download download_025_gfs.sh from GitHub to DOMAIN_DIR
echo "Downloading download_025_gfs.sh..."
curl -sSL https://raw.githubusercontent.com/goikelem/Automation-Automation-
Utility-Scripts/main/download_025_gfs.sh -o "$DOMAIN_DIR/download_025_gfs.sh"

# Download run_wrf.sh from GitHub to DOMAIN_DIR
echo "Downloading run_wrf.sh..."
curl -sSL https://raw.githubusercontent.com/goikelem/Automation-Automation-
Utility-Scripts/main/run_wrf.sh -o "$DOMAIN_DIR/run_wrf.sh"

# Make the downloaded scripts executable
chmod +x "$DOMAIN_DIR/download_025_gfs.sh" "$DOMAIN_DIR/run_wrf.sh"

# Define source and destination directories
SOURCE="$WRF_RUN"
DEST="$DOMAIN_DIR"

# Link all files and directories, including hidden entries
cp -as "$SOURCE"/. "$DEST"/

# --- Automatically update ~/.bashrc ---
BASHRC="$HOME/.bashrc"
echo "Updating $BASHRC with environment variables..."

# Add base WRF variables if they aren't already there
if ! grep -q "export WRF_HOME=" "$BASHRC"; then
    echo -e "\n# WRF Environment Variables" >> "$BASHRC"
    echo "export WRF_HOME=\"$WRF_HOME\"" >> "$BASHRC"
    echo "export WRF_WPS=\"$WRF_WPS\"" >> "$BASHRC"
    echo "export WRF_RUN=\"$WRF_RUN\"" >> "$BASHRC"
fi
```

```
# Update or add DOMAIN_DIR for this specific domain
if grep -q "export DOMAIN_DIR=" "$BASHRC"; then
    sed -i "s|export DOMAIN_DIR=.*|export DOMAIN_DIR=\"$DOMAIN_DIR\"|" "$BASHRC"
else
    echo "export DOMAIN_DIR=\"$DOMAIN_DIR\"" >> "$BASHRC"
fi

# Apply changes to the current terminal session immediately
source "$BASHRC"

# Verify the symbolic links and environment setup
ls -lah "$DEST"
echo "-------------------------------------------------------"
echo "Setup complete! All files and links are ready in: $DOMAIN_DIR"
echo "~/.bashrc has been updated with WRF variables and DOMAIN_DIR."
echo "-------------------------------------------------------"
```

Save the script as `setup_wrf_domain.sh` and, execute it

`chmod +x setup_wrf_domain.sh`

Then run it on your terminal as: `source ./setup_wrf_domain.sh`

```
(base) goitomk@gjob:~/book$ ./setup_wrf_domain.sh
Enter the domain name:
```

it waiting you input from your keyboards

Enter your desired name, e.g ethiowrf, mydomain ...

```
(base) goitomk@gjob:~/book$ ./setup_wrf_domain.sh
Enter the domain name: ethiowrf
```

Chapter two

2. Configure WRF domain

1. Method 1: The domain can be configured in two ways, directly from your browser (**https://goikelem.github.io/WRF-domain-Wizard/**)

2. Method 2: The second method is from your downloaded file inside the "`$WRF_HOME/domwizard/WRF-domain-Wizard.html`" file by running it from your terminal

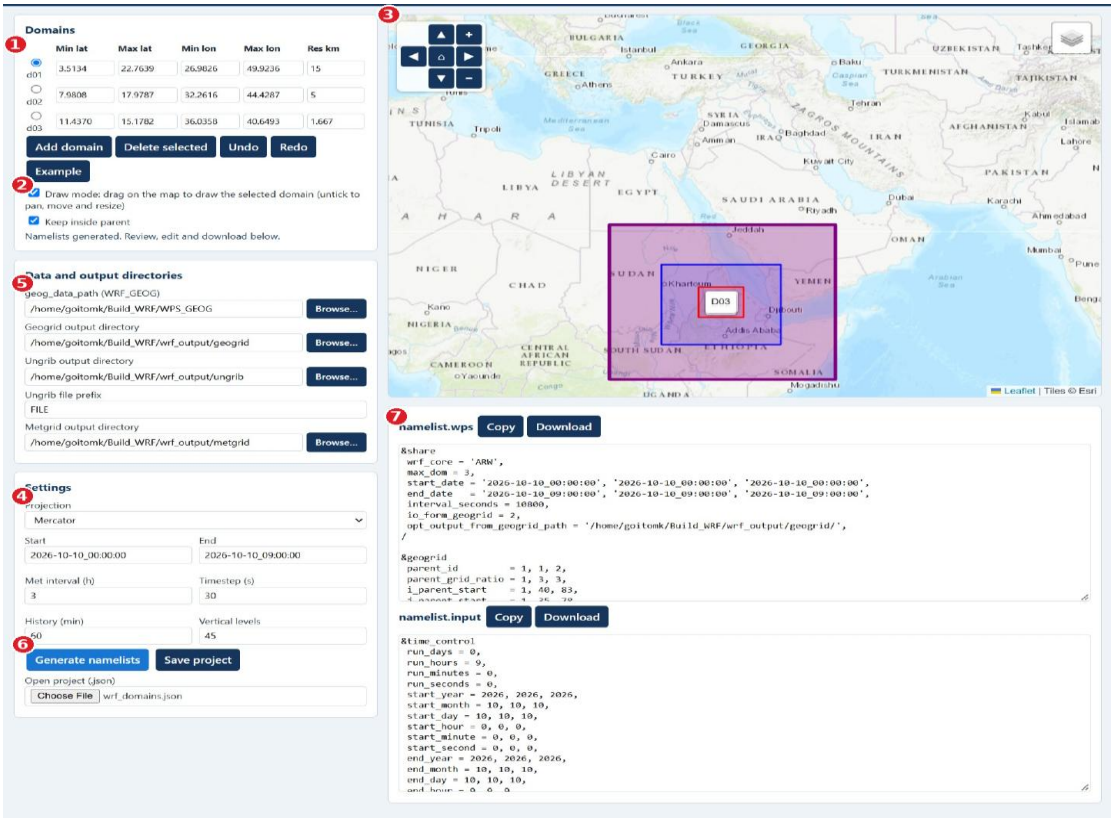
```
# Create domain from your terminal
# $WRF_HOME/domwizard/WRF-domain-Wizard.html

# For Linux:
xdg-open $WRF_HOME/domwizard/WRF-domain-Wizard.html

# For WSL on Windows:
explorer.exe $WRF_HOME/domwizard/WRF-domain-Wizard.html
```

2.1 WRF Domain Wizard: User Guide

This guide walks through the web app from an empty map to namelist.wps and namelist.input. Red numbers on each picture match the numbered points beside it.

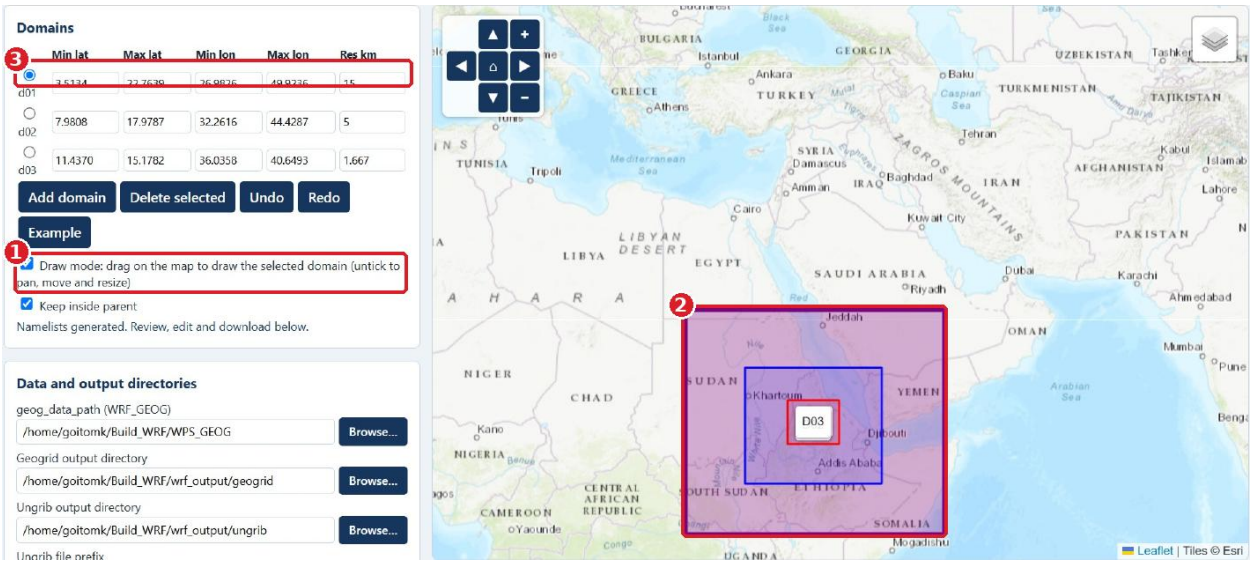


Before you start

- Open the live page (https://goikelem.github.io/WRF-domain-Wizard/). Chrome or Edge is recommended.
- An internet connection is needed for the map and tiles.
- Click Example at any time to load a 3-domain sample.
- Open this link and create the namelist.wps and namelist.input files inside the $WRF_HOME/ethiowrf directory

Step 1. Draw the outer domain (d01)

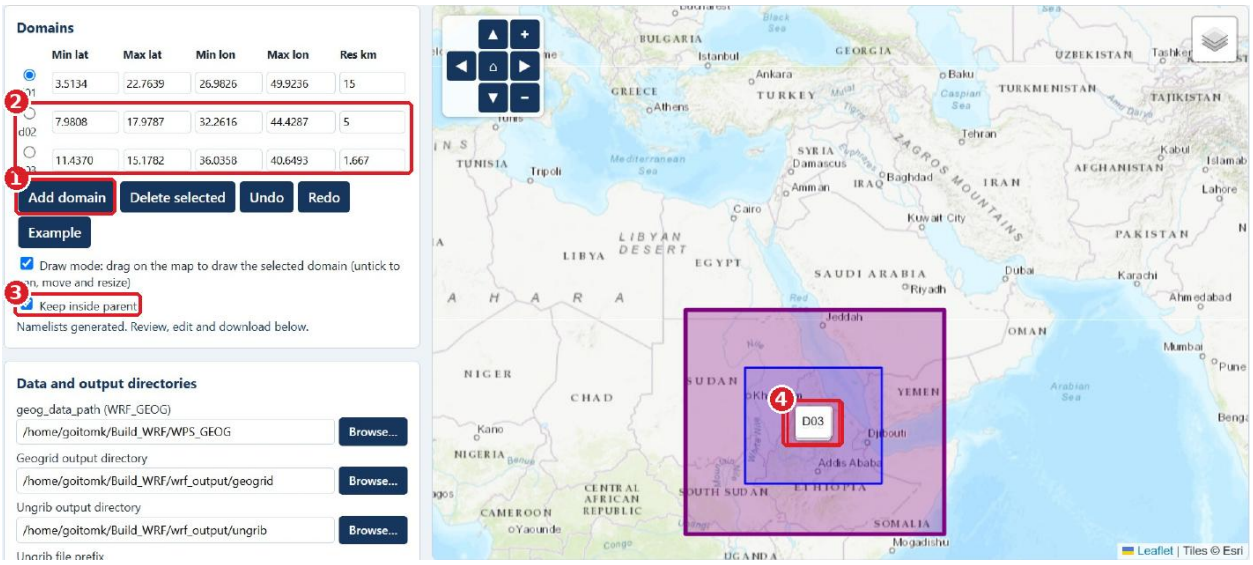
Draw mode is already on when the page opens, so you can start drawing straight away.



1. Draw mode: keep this box ticked. The cursor becomes a crosshair and dragging the map is switched off.
2. Drag on the map: press and hold at one corner of your area, move to the opposite corner and release. If you move the pointer close to the map edge while holding, the map scrolls by itself, so the far corner can be outside the current view.
3. Domains table: the d01 extents (Min/Max lat and lon) fill in automatically. You can also type them here, and set the resolution in km.

Step 2. Add nested domains (d02, d03, ...)

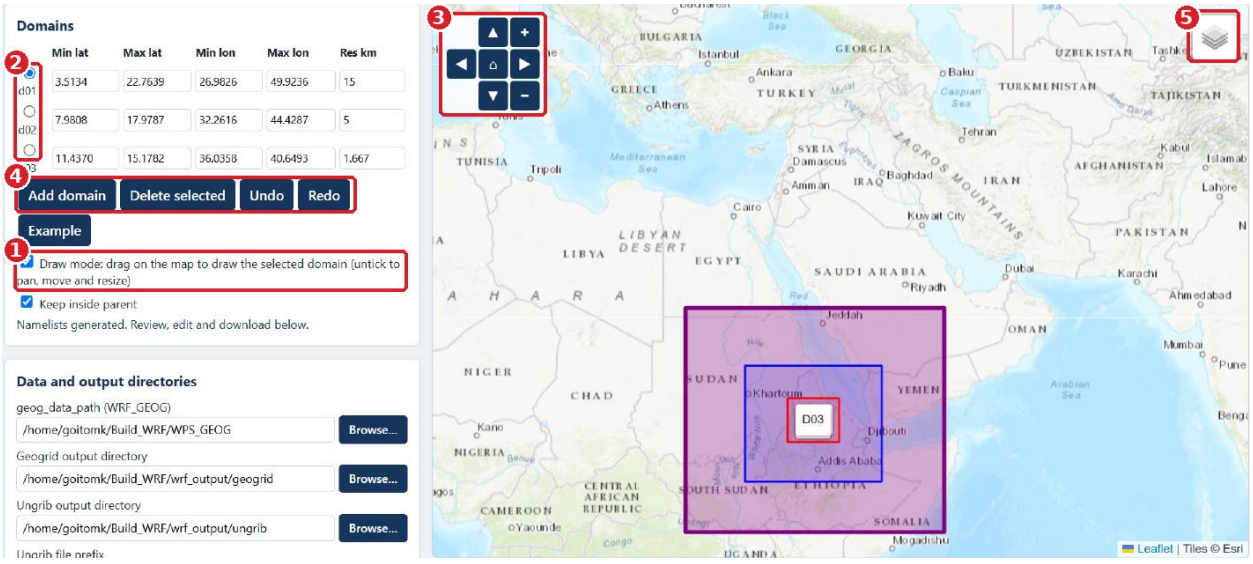
Each nest is drawn inside the previous one.



1. Add domain: adds a new row, up to 9 domains. The new resolution is one third of the previous one.
2. New rows (d02, d03): change the resolution if you need another ratio. The extents stay empty until you draw or type them.
3. Keep inside parent: stops a nest from being drawn or moved outside its parent box.
4. Draw the nest: drag a box inside its parent. The box is labelled D02, D03 and so on.

Step 3. Select, move and resize boxes

Untick Draw mode first. Then dragging works on the boxes and the map instead of drawing new ones.

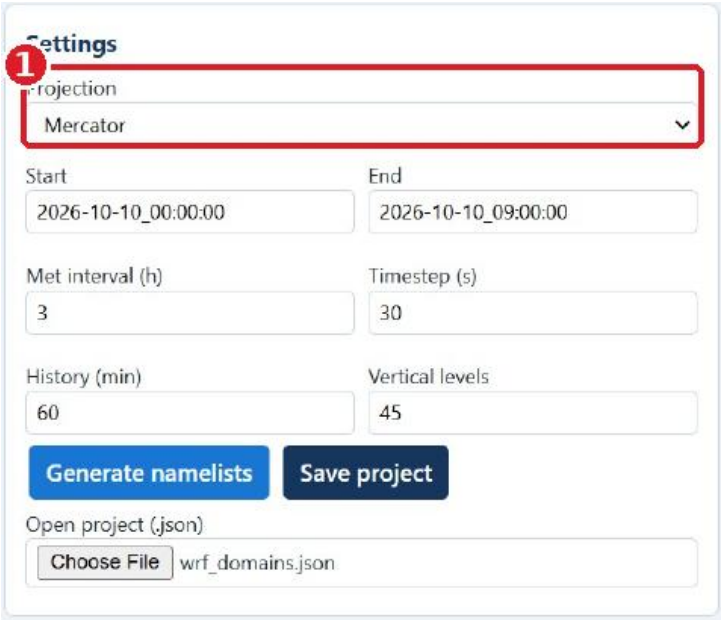


1. Draw mode: untick it (it is still ticked in this screenshot). Dragging the map now pans it.

2. Select a domain: tick its radio button, or click the box on the map. The selected box gets a thicker outline and white squares.
3. Navigation pad: the arrows pan the map, + and − zoom, and the house button fits all domains. With a box selected, drag its centre square to move it and a corner square to resize it.
4. Undo, Redo and Delete selected: step back or forward, or remove the selected domain (later ones are renumbered).
5. Layer switcher: the map starts on ESRI World Topo. Choose ESRI World Imagery for a satellite view.

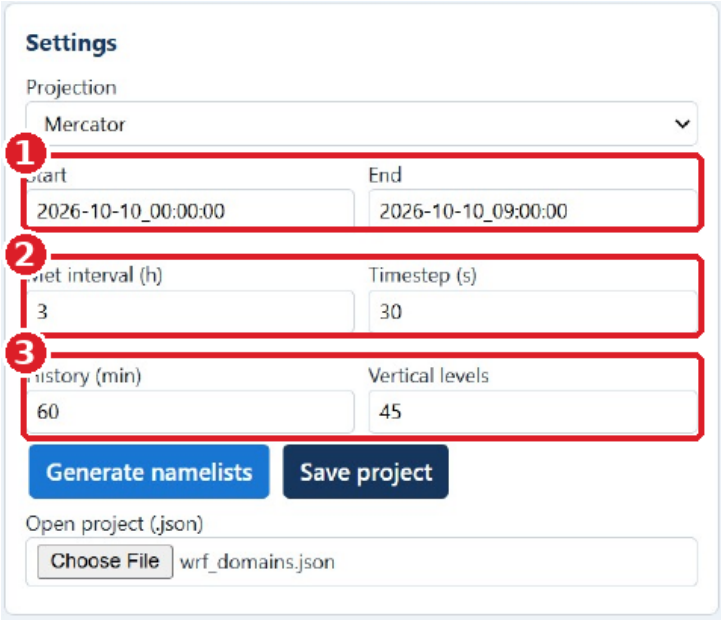
Step 4. Choose the projection

The reference point and true latitudes are calculated from d01.



1. Projection: choose Lambert Conformal, Mercator, Polar Stereographic or Latitude-Longitude. The screenshot uses Mercator.

Step 5. Set the timing and run options



1. Start and End: use the format YYYY-MM-DD_HH:MM:SS. End must be later than Start.
2. Met interval (h) and Timestep (s): how often your input data arrives, and the WRF time step.
3. History (min) and Vertical levels: output interval and number of vertical levels (e_vert).

Step 6. Set the data and output directories

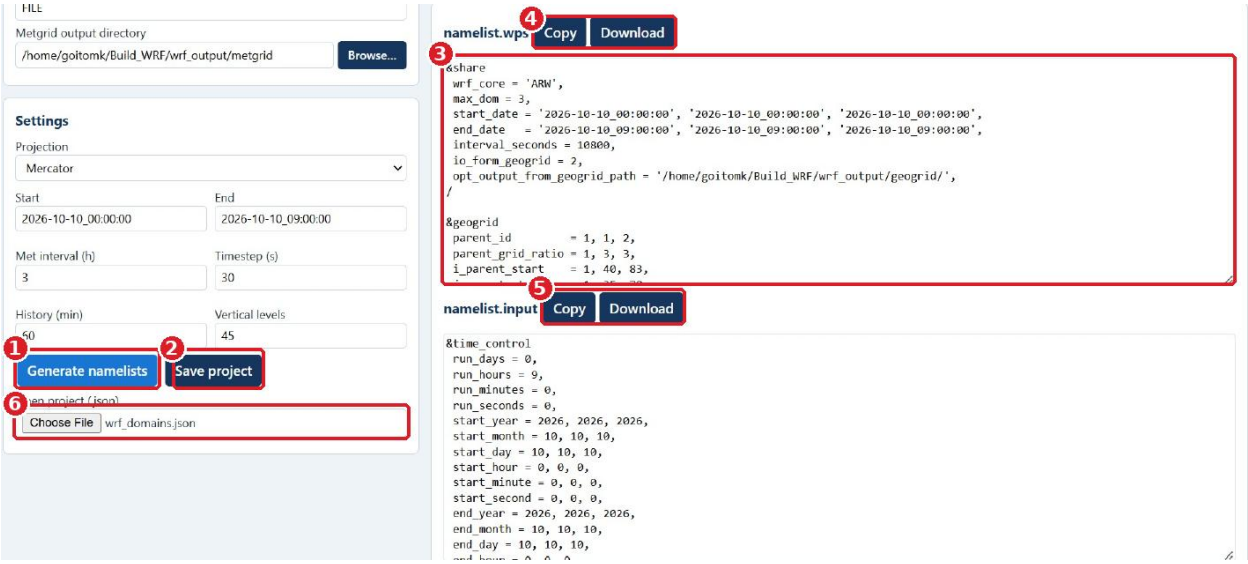


1. geog_data_path: your WRF_GEOG folder.
2. Geogrid output directory: where the geo_em files are written.
3. Ungrib output directory: where the intermediate files are written.
4. Ungrib file prefix: for example FILE. The namelist gets the directory and prefix joined, such as /home/user/wrf_output/ungrib/FILE.

5. Metgrid output directory: where the met_em files are written.
6. Browse buttons: pick a folder. Browsers hide full paths, so only the folder name replaces the last part of the existing path. Check the start of the path afterwards.

Step 7. Generate and export the namelists

The message under the domains table confirms success, or lists warnings if a nest does not fit inside its parent.



1. Generate namelists: builds namelist.wps and namelist.input from your domains and settings.
2. Save project: saves domains, settings and paths as a .json file.
3. Editors: read and edit the generated text before using it.
4. Copy and Download (namelist.wps): copy the text to the clipboard or save the file.
5. Copy and Download (namelist.input): the same for the WRF input namelist.
6. Open project: load a saved .json file to continue later.

Checking before a real run

- The grid sizes and parent start positions are estimated from the boxes you draw. Run geogrid.exe and plot the result to confirm the domains.
- e_we - 1 and e_sn - 1 of each nest must divide by its parent ratio. The tool aligns them, and warns about problems.
- The physics and dynamics values are general defaults. Adjust them for your case.

Troubleshooting

Problem	What to do
Map is blank	Check your internet connection, then hard-refresh with Ctrl+Shift+R.
Cannot drag the map	Untick Draw mode.
Cannot move or resize a box	Untick Draw mode and select the domain first.
"Cannot generate" message	Make sure every domain has numeric extents and a resolution, and that End is after Start.
Browse shows an upload prompt	Your browser lacks the folder picker. Nothing is uploaded; you can type the path instead.

Now we have created the **`namelist.wps`** and **`namelist.input`**. Originally, these files have to be inside the WPS-4.6.0 and WRF-ARW-4.6.0 directories, respectively; however, we are creating all important links to the **$WRF_HOME/ethiowrf** directory then all our tasks are done on the **ethiowrf** domain. Our **namelist.wps** file content is as below.

```
&share
 wrf_core = 'ARW',
 max_dom = 3,
 start_date = '2026-10-10_00:00:00', '2026-10-10_00:00:00', '2026-10-
10_00:00:00',
 end_date   = '2026-10-10_09:00:00', '2026-10-10_09:00:00', '2026-10-
10_09:00:00',
 interval_seconds = 10800,
 io_form_geogrid = 2,
 opt_output_from_geogrid_path = '/home/goitomk/Build_WRF/ethiowrf/',
/

&geogrid
 parent_id         = 1, 1, 2,
 parent_grid_ratio = 1, 3, 3,
 i_parent_start    = 1, 40, 83,
 j_parent_start    = 1, 35, 78,
 e_we              = 168, 265, 301,
 e_sn              = 144, 226, 253,
 geog_data_res     = 'default', 'default', 'default',
 dx = 15000,
 dy = 15000,
 map_proj = 'mercator',
 ref_lat = 13.13865,
 ref_lon = 38.45310,
 truelat1 = 13.13865,
```

```
 truelat2 = 0.00000,
 stand_lon = 38.45310,
 geog_data_path = '/home/goitomk/Build_WRF/WPS_GEOG',
/

&ungrib
 out_format = 'WPS',
 prefix = '/home/goitomk/Build_WRF/ethiowrf/FILE',
/

&metgrid
 fg_name = '/home/goitomk/Build_WRF/ethiowrf/FILE',
 io_form_metgrid = 2,
 opt_output_from_metgrid_path = '/home/goitomk/Build_WRF/ethiowrf/',
/
```

We will use the **namelist.wps** file in Chapter 3

Chapter three

WRF Modelling System and Preprocessing System

3.0 The WRF Modeling System Program Components

In general, the WRF system consists of a pre-processing component called WPS; the WRF-DA, which is an optional data assimilation component; the ARW solver, which is the key component of the modeling system that is composed of several initialization programs for `ideal`ized and `real`-data simulations, and the numerical integration program; and post-processing and visualization tools. The figure below provides a flow chart of the ARW core WRF modeling system.

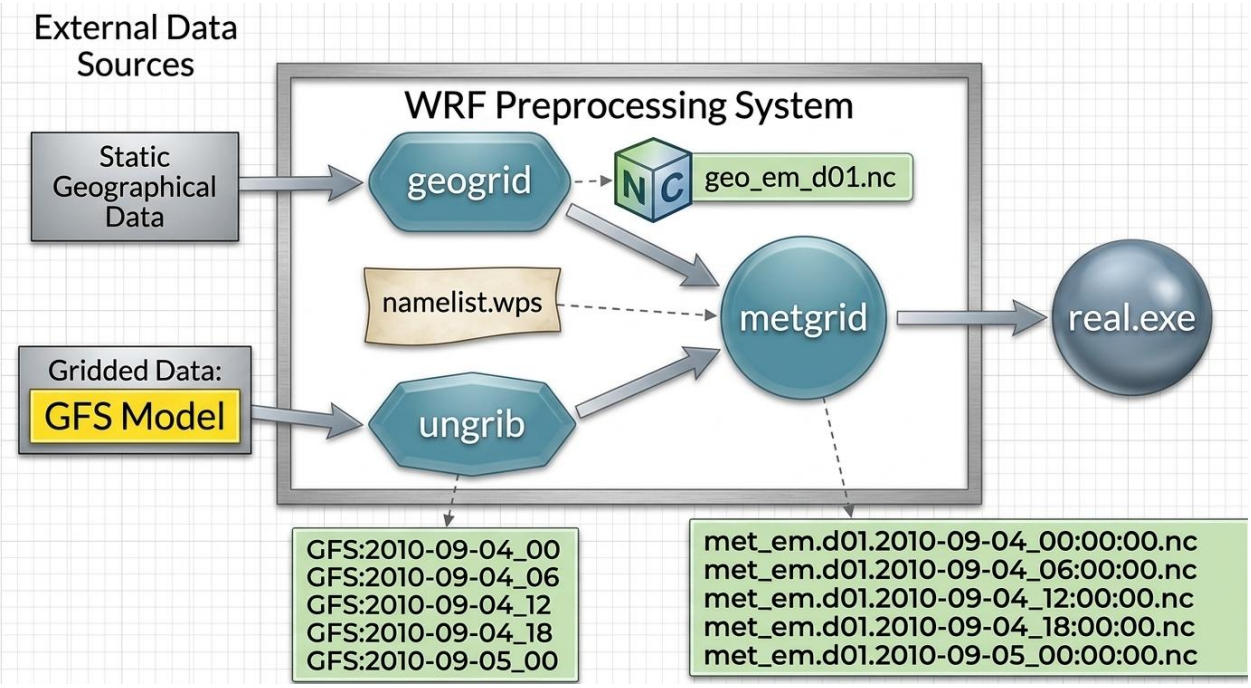


Figure 1: All three programs use parameters from the WPS namelist and shared namelist records in namelist.wps file to run.

3.1 WRF Preprocessing System (WPS)

The WPS component is mainly used for `real` data simulations. The primary functions of WPS are defining the spatial domain for the simulation; interpolating static terrestrial data (such as terrain, land use, and soils) to the defined spatial domain; and degribbing and interpolating meteorological data from another model's output to the spatial domain.

3.2 Spatial simulation domain for East Africa

The spatial domain for the Africa region includes nesting options over Eastern Ethiopia and the broader Ethiopia region. The parent domain has an 18 km spatial resolution, and both the nests have a 6 km and 2 km spatial resolution, respectively. The choice of the parent domain resolution covers the East Africa Region, whereas the child domains are based on the recommended parent-to-child spatial resolution ratio (1:3). The figure below shows the defined spatial simulation domain for East Africa (EA) with two nests. Note that changes in the simulation domain resolution will have a significant bearing on computing time.

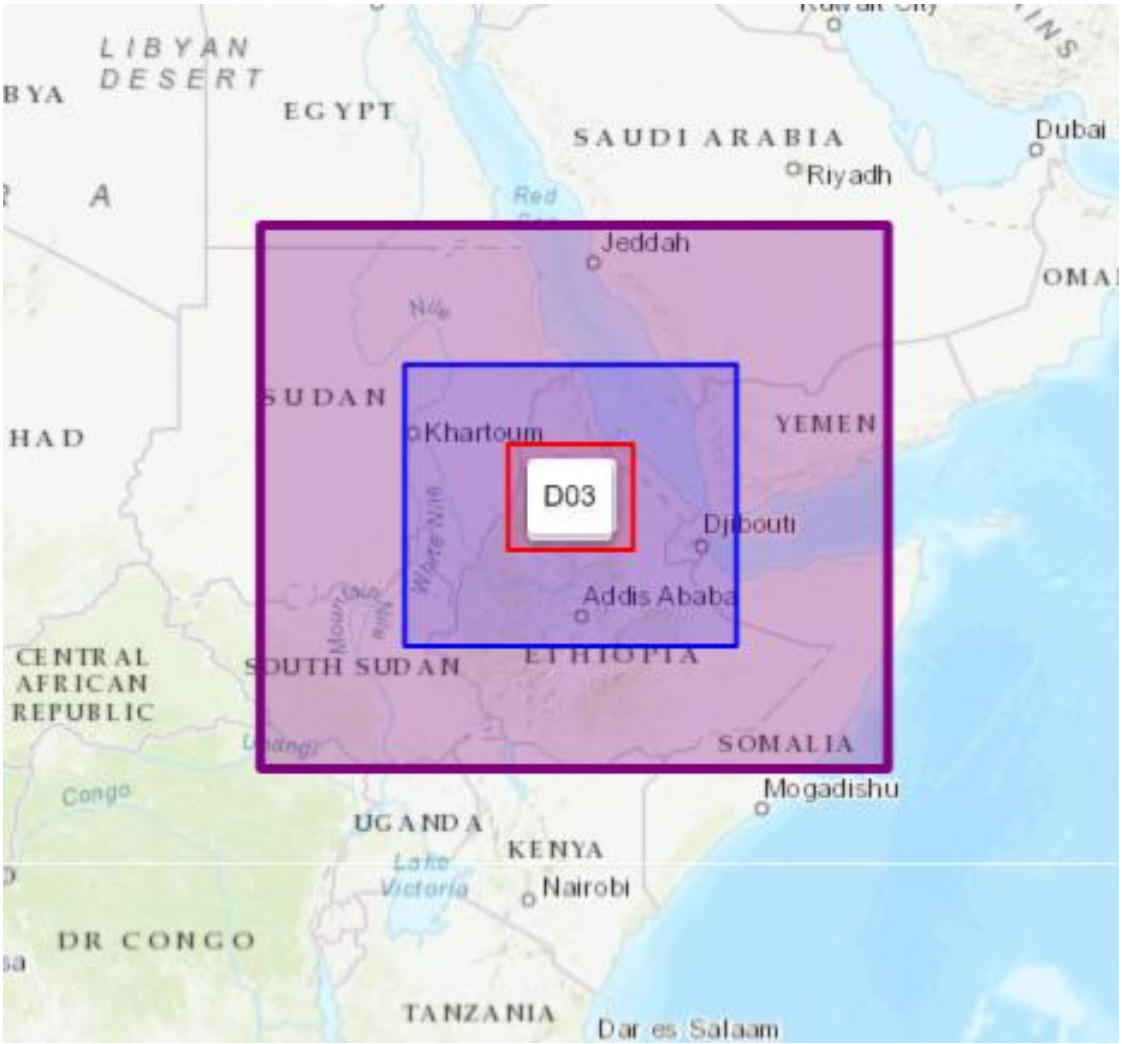


Figure 2. WRF Model domain configurations

3.3 Input gridded time-varying data for model run

The current setup of the WRF at ICS fetches input initial and lateral boundary condition data from the Global Forecasting System (GFS) of NCEP. The entire globe is covered by the GFS at a base horizontal resolution of 28 kilometres between grid points, which is used by operational forecasters

to predict weather out to 16 days in advance. However, due to the volume of data and the time required to download these datasets, the downloading script is set to download 27-kilometre resolution 3-hourly data for a 72-hour forecast. The total volume of data to be downloaded daily is 1.5 GB. Better resolution data can be used with an improved internet connection speed at ICS.

3.3.1 Static Geographical Data:

A complete set of data has been downloaded from NCAR which, when unzipped, takes nearly 52 GB. The NCAR site can be checked for updates at any time. The current model setup uses 5-minute resolution data for the parent model and 2-minute resolution data for the child domains, but the dataset has up to 30-second resolution available.

Learning Outcomes

This tutorial will enable you to develop new skills in applying experiment design techniques into practice. The following are the learning outcomes:

- You will understand the role of the WPS system.
- You will learn how to edit and change a **namelist.wps** file according to your experiment design needs.
- You will be able to plot your domain information using the NCAR Command Language (NCL) or Python (e.g., **wrf-python**) software.
- You will study how to create nested domains in WPS.

The new skills you will gain from this tutorial will enable you to create both simple and complex experiment designs using WPS. What you will learn here will also enable you to critically evaluate experiment designs from others and allow you to feel more confident when explaining the decisions you have made regarding your own design.

3.4 WRF Preprocessing System

The WRF Preprocessing System (WPS) is one of the most important components of WRF. It is through WPS that you will:

- Design your experiment.
- Prepare geographical information WRF needs.
- Convert meteorological data into a format WRF can use.
- Prepare data to be used by WRF.

Let's now learn how WPS actually works and about the different components that make up WPS: `geogrid`, `ungrib`, and `metgrid`.

3.4.1 WRF Preprocessing System

WPS is composed of three main programs:

- **geogrid.exe** - Related to processing geographical information based on the experiment design you have chosen.
- **ungrib.exe** - This program converts meteorological data from GRIB format into the so-called "WRF Intermediate Format" (WIF). WIF is a format that the next program, **metgrid.exe**, understands and is able to use.
- **metgrid.exe** - This program processes the WIF data and interpolates it horizontally. It then prepares the data to be used by WRF later on.

In this tutorial, we will be using the `geogrid.exe` program. You will learn how to work with the other programs later on in this course.

Now that you know a little bit about the different WPS programs, there is one thing missing: How will you communicate with these programs?

*The answer to this question is very important. In order for us to communicate with each of the WPS programs, we need to work with a file called **namelist.wps** - it is this file that provides the connection between what we want (in terms of experiment design) and what WPS should do. We will be learning about namelist.wps in this tutorial as well.*

So, to summarise, we can make an analogy between WPS and preparing some food. Before you can prepare any food, you will need to gather the necessary ingredients and "preprocess" them. Although the above analogy is simple, it is used to help you remember the function WPS and its `namelist.wps` file. Without WPS, we cannot run the WRF model. This is why it is important for us to understand how it works and to be able to use it properly, so that our experiment design can be implemented successfully.

The WRF Preprocessing System (WPS) is a set of three programs whose collective role is to prepare input to the `real` program for `real`-data simulations. Each of the programs performs one stage of the preparation: `geogrid` defines model domains and interpolates static geographical data to the grids; `ungrib` extracts meteorological fields from GRIB-formatted files; and `metgrid`

horizontally interpolates the meteorological fields extracted by **ungrib** to the model grids defined by **geogrid**.

As the GFS data contains more variables required by WRF, **ungrib** uses a predefined list of variables maintained in tables called **Vtables**. If input data sources other than GFS are used, appropriate **Vtables** should be selected. The **metgrid** program horizontally interpolates time-variant meteorological data decoded by the **ungrib** program to the simulation domains defined by the **geogrid** program. The way **metgrid** interpolates meteorological data is controlled by methods defined in **METGRID.TBL**. The output from the **metgrid** process can then be ingested by the WRF **real** program. The figure below shows the data flow between different programs of WPS.

3.5 Prepare input data for the WRF model

This shell script is used to extract and download GFS 0.25-degree sub-region datasets from the NOAA NOMADS server. This script is engineered specifically to streamline numerical weather prediction (NWP) workflows, such as WRF preprocessing via the WPS (WRF Preprocessing System).

```
#!/bin/bash
# ------------------------------------------------------------------------
# Download GFS 0.25 deg (gfs.t00z.pgrb2.0p25) for a sub-region from NOMADS,
# keeping only the variables/levels required by Vtable_0.25.GFS.
#
# Usage:  ./download_gfs_025.sh [YYYYMMDD] [START_HR] [END_HR] [STEP_HR]
# Example: ./download_gfs_025.sh 20261011 0 72 3
# Defaults: today (UTC), 0, 120, 3
# ------------------------------------------------------------------------

# ---------------- User settings ----------------
CYCLE="00"                  # model cycle (t00z)
LEFTLON=20                  # western bound  (deg E)
RIGHTLON=54                 # eastern bound  (deg E)
TOPLAT=25                   # northern bound (deg N)
BOTTOMLAT=-10               # southern bound (deg N, negative = south)
OUTDIR="$WRF_HOME/GFS_Input"
# -----------------------------------------------

DATE="${1:-$(date -u +%Y%m%d)}"
START="${2:-0}"
END="${3:-120}"
STEP="${4:-3}"

BASE="https://nomads.ncep.noaa.gov/cgi-bin/filter_gfs_0p25.pl"

# Variables selected from the Vtable (13)
VARS="HGT ICEC LAND MSLET PRES RH SNOD SOILW TMP TSOIL UGRD VGRD WEASD"
```

```
# 41 isobaric levels (hPa) + surface-type levels
PLEVS="1000 975 950 925 900 850 800 750 700 650 600 550 500 450 400 350 300 250
200 150 100 70 50 40 30 20 15 10 7 5 3 2 1 0.7 0.4 0.2 0.1 0.07 0.04 0.02 0.01"
OTHER_LEVS="2_m_above_ground 10_m_above_ground surface mean_sea_level \
0-0.1_m_below_ground 0.1-0.4_m_below_ground 0.4-1_m_below_ground 1-
2_m_below_ground \
max_wind tropopause"

# Build the query-string pieces once
QS=""
for v in $VARS; do QS+="&var_${v}=on"; done
for l in $PLEVS; do QS+="&lev_${l}_mb=on"; done
for l in $OTHER_LEVS; do QS+="&lev_${l}=on"; done
QS+="&subregion=&leftlon=${LEFTLON}&rightlon=${RIGHTLON}&toplat=${TOPLAT}&bottoml
at=${BOTTOMLAT}"
QS+="&dir=%2Fgfs.${DATE}%2F${CYCLE}%2Fatmos"

mkdir -p "$OUTDIR" || exit 1
cd "$OUTDIR" || exit 1

echo "GFS ${DATE} ${CYCLE}Z  f$(printf %03d $START)-f$(printf %03d $END) every
${STEP} h"
echo "Region: ${LEFTLON}E-${RIGHTLON}E, ${BOTTOMLAT}-${TOPLAT}N  ->  $OUTDIR"

FAILED=0
for ((h=START; h<=END; h+=STEP)); do
    FHR=$(printf "%03d" "$h")
    FILE="gfs.t${CYCLE}z.pgrb2.0p25.f${FHR}"
    OUT="${DATE}${CYCLE}_${FILE}"       # e.g.
2026101100_gfs.t00z.pgrb2.0p25.f000

    if [[ -s "$OUT" ]]; then
        echo "[skip] $OUT already exists"
        continue
    fi

    echo "[get ] $OUT"
    curl -sS -f --retry 5 --retry-delay 10 --connect-timeout 30 \
         -o "${OUT}.part" "${BASE}?file=${FILE}${QS}"

    # A valid GRIB2 file starts with "GRIB"; otherwise NOMADS returned an error
page
    if [[ -s "${OUT}.part" && "$(head -c 4 "${OUT}.part")" == "GRIB" ]]; then
        mv "${OUT}.part" "$OUT"
```

```
    else
        echo "[FAIL] $OUT (data not yet available or request error)"
        rm -f "${OUT}.part"
        FAILED=$((FAILED+1))
    fi
    sleep 2        # be polite to NOMADS
done

echo "Done. Failed files: $FAILED"

# ------------------------------------------------------------------------
```

Save the script as `download_025_gfs.sh` and execute it inside the $WRF_RUN directory

```
chmod +x download_025_gfs.sh
```

- **Bandwidth & Storage Efficiency:** Subsets global GFS 0.25° data directly on the server side prior to downloading, saving gigabytes of transfer bandwidth and storage space.
- **Targeted Extraction:** Dynamically constructs query parameters to pull the exact 13 variables across 41 isobaric levels and surface layers required by Vtable_0.25.GFS.
- **Robust Error Handling:** Features automated file existence checks to skip redundant downloads, robust curl retry logic, and header inspection ('GRIB') to validate data integrity.

2. Prerequisites

Ensure your execution environment satisfies the following dependencies before running the script:

- **Operating System:** Linux, macOS, or Windows Subsystem for Linux (WSL)
- **Shell Environment:** Bash shell environment (version 4.0 or higher recommended)
- **Networking Utility:** curl command-line tool installed and accessible in standard PATH
- **Network Access:** HTTP/HTTPS connection allowed to nomads.ncep.noaa.gov

3. Customization & User Settings

Modify the configuration header inside the script using any plain text editor to define your region of interest and local directory structure:

```
# ---------------- User settings ----------------
CYCLE="00"                 # Model cycle (e.g., "00", "06", "12", "18")
LEFTLON=20                 # Western boundary (degrees East)
RIGHTLON=54                # Eastern boundary (degrees East)
TOPLAT=25                  # Northern boundary (degrees North)
BOTTOMLAT=-10              # Southern boundary (degrees North, negative = South)
OUTDIR="$WRF_HOME/GFS_025"     # Local directory to store downloaded GRIB2 files
# -----------------------------------------------
```

4. Usage & Execution Options

The script accepts up to four optional positional arguments. If omitted, default values are automatically assigned:

```
./download_gfs_025.sh [YYYYMMDD] [START_HR] [END_HR] [STEP_HR]
```

Default Parameter Behaviors

- **Date (YYYYMMDD):** Current UTC date calculated dynamically via $(date -u +%Y%m%d)
- **Start Hour:** 0 (Initial forecast / analysis state f000)
- **End Hour:** 120 (5-day forecast horizon)
- **Step Hour:** 3 (Interval between successive output files in hours)

Execution Examples

- **Default Run:** Runs using today's date (UTC) from hour 0 to 120 at 3-hour intervals: ./download_gfs_025.sh
- **Custom Date & Range:** Downloads data for Oct 11, 2026, covering 0 to 72 hours every 3 hours: ./download_gfs_025.sh 20261011 0 72 3

5. Step-by-Step Technical Execution Workflow

Extracted Variables & Levels Overview

Category	Selected Parameters / Range
Variables (13)	HGT, ICEC, LAND, MSLET, PRES, RH, SNOD, SOILW, TMP, TSOIL, UGRD, VGRD, WEASD
Isobaric Levels (41)	1000, 975, 950 ... 1, 0.7, 0.4, 0.2, 0.1, 0.07, 0.04, 0.02, 0.01 hPa
Surface & Other Levels	2_m_above_ground, 10_m_above_ground, surface, mean_sea_level, soil layers (0-0.1m, 0.1-0.4m, 0.4-1m, 1-2m), max_wind, tropopause

- **1. Parameter Resolution:** Evaluates positional inputs and sets default parameters for missing arguments.
- **2. Query String Building:** Constructs a parameterized URL payload (&var_*, &lev_*, bounding coordinates) pointing to filter_gfs_0p25.pl.
- **3. Existing File Check:** Iterates through forecast steps (e.g. f000, f003) and verifies if the output file already exists to avoid redundant network traffic.
- **4. Resilient Fetching:** Invokes curl with --retry 5 and --connect-timeout 30 to download data into a temporary .part file.
- **5. Header Verification:** Inspects the first 4 bytes using head -c 4. If equal to 'GRIB', the file is renamed to its final target name. Otherwise, it is discarded as incomplete/error page.
- **6. Server Courtesy:** Executes sleep 2 after each iteration to maintain server compliance and prevent rate limiting.

6. Integration with WRF Preprocessing System (WPS)

After successfully downloading the GRIB2 sub-region files, execute the following standard WPS commands in your WPS working directory to prepare meteorological fields for WRF:

This sequence of commands represents the standard WPS (WRF Preprocessing System) workflow. It prepares your geographic domain and meteorological data (in this case, GFS forecasts) so they can be processed by the WRF numerical model using your ethiowrf domain as an example.

Step 1: Navigate to the Domain Directory

```
cd $DOMAIN_DIR
```

Explanation: Changes your current working terminal directory to the specific simulation folder (e.g., `$HOME/Build_WRF/ethiowrf`), ensuring all subsequent preprocessing commands execute in the correct workspace.

Verification: Run pwd and confirm the path ends with ethiowrf.

Step 2: Run Geogrid

```
./geogrid.exe
```

Explanation: Executes the geogrid program. It reads the domain configuration settings in **namelist.wps** and interpolates static geographical datasets (topography, land use, soil categories, vegetation fraction) onto your model grid.

Verification: Check that files named **geo_em.d01.nc** (and additional domains if nested) are created, and inspect **geo_em_log.0000** to verify it ends with SUCCESS COMPLETE.

Step 3: Link GRIB Files

```
./link_grib.csh $DOMAIN_DIR/GFS_Input/2026101000_gfs.t00z.pgrb2.0p25.f00*
```

This one is for the experiment to check, using limited data, if you need to run through your time step; link your data to the entire directory. `./link_grib.csh $DOMAIN_DIR/GFS_Input/*`

Explanation: Uses WPS's helper script (**link_grib.csh**) to create sequential symbolic links (GRIBFILE.AAA, GRIBFILE.AAB, etc.) pointing to your raw GFS forecast files stored in the GFS_Input directory. ungrib requires this naming convention to process time-series data chronologically.

Verification: Run ls -l GRIBFILE.* to confirm the symbolic links point correctly to your raw GFS input files.

Step 4: Link the Variable Table

```
ln -sf ungrib/Variable_Tables/Vtable.GFS Vtable
```

Explanation: Creates a symbolic link named Vtable that points specifically to the GFS variable definition table (**Vtable.GFS**). This tells ungrib how to decode the variable names and meteorological fields inside your GFS GRIB files.

Verification: Run **`ls -l Vtable`** and verify it points to **`ungrib/Variable_Tables/Vtable.GFS`**.

Step 5: Run Ungrib

```
./ungrib.exe >& log.ungrib
```

Explanation: Executes the ungrib program to unpack the raw GRIB meteorological data and convert it into intermediate format files (FILE:YYYY-MM-DD_HH). It redirects both standard output and error logs into a file called log.ungrib.

Verification: Check that intermediate files (e.g., **FILE:2026-10-10_00)** have been generated, and run **tail -n 20 log.ungrib** to ensure it finished with Successful completion.

Step 6: Run Metgrid

```
./metgrid.exe
```

Explanation: Executes the final WPS step, metgrid. It takes the static terrestrial fields from geogrid (**geo_em.d01.nc**) and the unpacked meteorological fields from ungrib, and horizontally interpolates the weather data onto your domain grid.

Verification: Check for the creation of met_**em.d01.2026-10-10_00:00:00.nc** files and confirm metgrid.log reports successful completion.

3.4.2 WRF Model

The WRF model (v4.6.0) has initialization (`ideal.exe` and `real.exe`), numerical integration (`wrf.exe`), one-way nesting (`ndown.exe`), and tropical storm bogussing (`tc.exe`) programs.

The initialization programs are called `ideal` and `real`. Ideal simulation allows users to run controlled scenarios and does not require external input data. It is also used to test whether the model programs can correctly run on a particular architecture and compiler. The `real` program needs `real`-data cases, which require pre-processing from the WPS package. The output data from the `metgrid` program run are used as input to the `real` program. The main functions of the `real` initialization program are:

- Creates initial and boundary condition files for **real**-data cases.
- Does vertical interpolation to model levels (when using WPS).
- Does vertical dynamic (hydrostatic) balance.
- Does soil vertical interpolations and land-use mask checks.

The WRF numerical integration program (`wrf.exe`) performs time integration, uses initial and boundary conditions generated by the `real` program, and runs the model simulation with run-time selected namelist options such as model dynamics, physics choices, timestep, length of simulation, etc.

For a complete description of the model, readers are advised to read the WRF Version 4.6.0 User's Guide online: https://www2.mmm.ucar.edu/wrf/users/docs/user_guide_v4/v4.6/ The PDF version is available on the WRF model implementation media.

Running WRF in real data cases requires the successful running of WPS pre-processing programs. Follow the following steps to run WRF for real data cases: Before you run WRF, you need to edit the namelist.input file (you may refer to README.namelist in the WRF-4.6.0-ARW/run directory) to change the length of the forecast, frequency of output, domain size, timestep, physics options, dynamic options, etc. The contents of the **namelist.input** for `ethiowrf` use are listed below:

```
&time_control
 run_days = 0,
 run_hours = 9,
 run_minutes = 0,
 run_seconds = 0,
 start_year = 2026, 2026, 2026,
 start_month = 10, 10, 10,
 start_day = 10, 10, 10,
 start_hour = 0, 0, 0,
 start_minute = 0, 0, 0,
 start_second = 0, 0, 0,
 end_year = 2026, 2026, 2026,
 end_month = 10, 10, 10,
 end_day = 10, 10, 10,
 end_hour = 9, 9, 9,
 end_minute = 0, 0, 0,
```

```
 end_second = 0, 0, 0,
 interval_seconds = 10800,
 input_from_file = .true., .true., .true.,
 history_interval = 60, 60, 60,
 frames_per_outfile = 1,
 restart = .false.,
 restart_interval = 1440,
 io_form_history = 2,
 io_form_restart = 2,
 io_form_input = 2,
 io_form_boundary = 2,
/

&domains
 time_step = 30,
 time_step_fract_num = 0,
 time_step_fract_den = 1,
 max_dom = 3,
 e_we = 168, 265, 301,
 e_sn = 144, 226, 253,
 e_vert = 45, 45, 45,
 p_top_requested = 5000,
 num_metgrid_levels = 34,
 num_metgrid_soil_levels = 4,
 dx = 15000, 5000, 1667,
 dy = 15000, 5000, 1667,
 grid_id = 1, 2, 3,
 parent_id = 1, 1, 2,
```

```
 i_parent_start = 1, 40, 83,
 j_parent_start = 1, 35, 78,
 parent_grid_ratio = 1, 3, 3,
 parent_time_step_ratio = 1, 3, 3,
 feedback = 1,
 smooth_option = 0,
/

&physics
 mp_physics = 8, 8, 8,
 ra_lw_physics = 4, 4, 4,
 ra_sw_physics = 4, 4, 4,
 bl_pbl_physics = 1, 1, 1,
 sf_sfclay_physics = 1, 1, 1,
 sf_surface_physics = 2, 2, 2,
 cu_physics = 1, 0, 0,
/

&dynamics
 hybrid_opt = 2,
 w_damping = 1,
 diff_opt = 1,
 km_opt = 4,
 damp_opt = 3,
 zdamp = 5000.,
 dampcoef = 0.2,
 non_hydrostatic = .true.,
 moist_adv_opt = 1,
```

```
 scalar_adv_opt = 1,
/

&bdy_control
 spec_bdy_width = 5,
 spec_zone = 1,
 relax_zone = 4,
 specified = .true.,
 nested = .true.,
/

&namelist_quilt
 nio_tasks_per_group = 0,
 nio_groups = 1,
/
```

Make sure you metgirb output file exists in the ethionet directory.

Run the real data initialization program. The command for running this may vary depending on your system and the number of processors you have available. In my case, I want to use 2 processors and 8 cores; you may choose to issue the command as follows:

Step 1: Run real

```
mpirun -np 2 ./real.exe
```

Successful completion of the real program run have "real_em: SUCCESS EM_REAL INIT" printed at the end of the rsl.error.0000 file. From the ethiowrf directory, open rsl.error.0000 using a text editor to check. It should also produce wrfinput_d01 and wrfbdy_d01 files. Finally, run the wrf.exe to produce WRF output data.

Step 2: Run wrf

```
mpirun -np 2 ./wrf.exe
```

Successful completion of the WRF model program run has "wrf: SUCCESS COMPLETE WRF" printed at the end of the rsl.error.0000 or rsl.out.000 files. Note that the error/output log files are created for each parallel run. The model run produces the following files, which correspond to the date on which the model is run:

```
wrfout_d01_YYYY-MM-DD_00:00:00.nc
wrfout_d02_YYYY-MM-DD_00:00:00.nc
wrfout_d03_YYYY-MM-DD_00:00:00.nc
```

After the model is run and produces the above outputs, the model run automation script (will be described below) changes the file name as listed below. Windows will not process the above naming because of the special characters in the file names. The final renamed output files are stored under `$WRF_HOME/ethiowrf/wrf_output`/YYYYMMDD00.

```
wrfout_d01_YYYY_MM_DD_00.nc
wrfout_d02_YYYY_MM_DD_00.nc
wrfout_d03_YYYY_MM-DD_00.nc
```

```
!/usr/bin/env bash

# ==============================================================================
# WRF/WPS run script
# Author: Goitom Kelem
#
# Usage:
#   ./run_wrf.sh --auto
#   ./run_wrf.sh YYYYMMDD CYCLE FORECAST_HOURS
#
# Examples:
#   ./run_wrf.sh --auto
#   ./run_wrf.sh 20261016 00 72
#
# Required environment:
#   WRF_HOME=/path/to/WRF installation root
#
# Expected layout (adjust WRF_HOME below or export it):
#   $WRF_HOME/ethiowrf/             WPS executables and namelist.wps
```

```
#   $WRF_HOME/ethiowrf/             WRF executables and namelist.input
#   $WRF_HOME/ethiowrf/GFS_InputYYYYMMDDCC/ GRIB input files
#   $WRF_HOME/scripts/              this script
# ============================================================================

set -Eeuo pipefail
IFS=$'\n\t'

die() { printf 'ERROR: %s\n' "$*" >&2; exit 1; }
log() { printf '\n[%s] %s\n' "$(date -u '+%Y-%m-%d %H:%M:%S UTC')" "$*"; }
require_file() { [[ -f "$1" ]] || die "Required file not found: $1"; }
require_executable() { [[ -x "$1" ]] || die "Required executable not found or not
executable: $1"; }

usage() {
    cat <<'USAGE'
Usage:
  run_wrf.sh --auto
  run_wrf.sh YYYYMMDD CYCLE FORECAST_HOURS

Arguments:
  YYYYMMDD        Forecast initialization date in UTC, e.g. 20261016
  CYCLE           00, 06, 12, or 18 UTC
  FORECAST_HOURS  Integer from 1 to 72

Environment:
  WRF_HOME        WRF installation root; must contain ethiowrf/ and scripts/
  REAL_NP         MPI ranks for real.exe (default: 2)
  WRF_NP          MPI ranks for wrf.exe  (default: 8)
USAGE
}

# Find installation root. Export WRF_HOME before running for predictable
behavior.
WRF_HOME="${WRF_HOME:-${HOME}/Build_WRF}"
BASE_DIR="${WRF_HOME}/ethiowrf"
SCRIPT_DIR="${WRF_HOME}/scripts"
REAL_NP="${REAL_NP:-2}"
WRF_NP="${WRF_NP:-8}"

[[ -d "$BASE_DIR" ]] || die "Run directory does not exist: $BASE_DIR"
[[ "$REAL_NP" =~ ^[1-9][0-9]*$ ]] || die "REAL_NP must be a positive integer."
[[ "$WRF_NP" =~ ^[1-9][0-9]*$ ]] || die "WRF_NP must be a positive integer."

MODE="case"
```

```
if [[ $# -eq 1 && "$1" == "--auto" ]]; then
    MODE="auto"
elif [[ $# -ne 3 ]]; then
    usage
    exit 2
fi

if [[ "$MODE" == "auto" ]]; then
    # Automatically initialize at today's 00 UTC cycle.
    inDate="$(date -u +%Y%m%d)"
    cc="00"
    fc="72"
else
    inDate="$1"
    cc="$2"
    fc="$3"
fi

[[ "$inDate" =~ ^[0-9]{8}$ ]] || die "Date must be YYYYMMDD."
[[ "$cc" =~ ^(00|06|12|18)$ ]] || die "Cycle must be 00, 06, 12, or 18."
[[ "$fc" =~ ^[0-9]+$ ]] || die "Forecast length must be an integer from 1 to 72."
(( fc >= 1 && fc <= 72 )) || die "Forecast length must be between 1 and 72
hours."
date -u -d "${inDate} ${cc}:00:00" '+%Y%m%d%H' >/dev/null 2>&1 || die "Invalid
date/cycle: ${inDate} ${cc}."

start_epoch="$(date -u -d "${inDate} ${cc}:00:00" +%s)"
end_epoch=$(( start_epoch + fc * 3600 ))
sYear="$(date -u -d "@${start_epoch}" +%Y)"
sMonth="$(date -u -d "@${start_epoch}" +%m)"
sDay="$(date -u -d "@${start_epoch}" +%d)"
sHour="$(date -u -d "@${start_epoch}" +%H)"
eYear="$(date -u -d "@${end_epoch}" +%Y)"
eMonth="$(date -u -d "@${end_epoch}" +%m)"
eDay="$(date -u -d "@${end_epoch}" +%d)"
eHour="$(date -u -d "@${end_epoch}" +%H)"

cDate="${inDate}${cc}"
gribdir="${BASE_DIR}/GFS_Input/${cDate}"
wrfdir="${BASE_DIR}/wrf_output/${cDate}"
mkdir -p "$wrfdir"

start_stamp="${sYear}-${sMonth}-${sDay}_${sHour}:00:00"
end_stamp="${eYear}-${eMonth}-${eDay}_${eHour}:00:00"
start_date="start_date = '${start_stamp}'"
```

```
end_date="end_date = '${end_stamp}'"

wps_namelist="${BASE_DIR}/namelist.wps"
WRF_RUN_DIR="${WRF_RUN_DIR:-$BASE_DIR}"
wrf_namelist="${WRF_RUN_DIR}/namelist.input"
require_file "$wps_namelist"
require_file "$wrf_namelist"
require_executable "${BASE_DIR}/geogrid.exe"
require_executable "${BASE_DIR}/ungrib.exe"
require_executable "${BASE_DIR}/metgrid.exe"
require_executable "${BASE_DIR}/real.exe"
require_executable "${BASE_DIR}/wrf.exe"
require_executable "${BASE_DIR}/link_grib.csh"
[[ -d "$gribdir" ]] || die "GRIB input directory not found: $gribdir"

run_start="$(date -u +%s)"
log "WRF run started. Date=${inDate}, cycle=${cc} UTC, forecast=${fc} hours"
log "Input GRIB directory: $gribdir"
log "Output directory: $wrfdir"

# Back up namelists before editing them.
cp -p "$wps_namelist" "${wps_namelist}.bak.$(date -u +%Y%m%dT%H%M%S)"
cp -p "$wrf_namelist" "${wrf_namelist}.bak.$(date -u +%Y%m%dT%H%M%S)"

# Replace a namelist assignment while preserving the number of comma-separated
# values already present (supports d01...d0N rather than assuming 3 domains).
set_namelist_value() {
    local file="$1" key="$2" value="$3"
    KEY="$key" VALUE="$value" FILE="$file" python3 - <<'PY'
import os, re, sys
path, key, value = os.environ["FILE"], os.environ["KEY"], os.environ["VALUE"]
with open(path, "r", encoding="utf-8") as f:
    lines = f.readlines()
found = False
pat = re.compile(r"^(\s*" + re.escape(key) +
r"\s*=\s*)(.*?)(\s*(?:!.*)?)(\r?\n?)$", re.I)
for i, line in enumerate(lines):
    m = pat.match(line)
    if not m:
        continue
    old = m.group(2).strip()
    # Keep the existing number of domain entries where possible.
    count = max(1, old.count(",") + (0 if old.endswith(",") else 1))
    vals = ", ".join([value] * count)
    if old.endswith(","):
```

```
        vals += ","
    lines[i] = m.group(1) + vals + m.group(3) + m.group(4)
    found = True
    break
if not found:
    print(f"ERROR: namelist key '{key}' not found in {path}", file=sys.stderr)
    sys.exit(3)
with open(path, "w", encoding="utf-8") as f:
    f.writelines(lines)
PY
}

log "Updating namelist.wps dates"
set_namelist_value "$wps_namelist" start_date "'${start_stamp}'"
set_namelist_value "$wps_namelist" end_date "'${end_stamp}'"

# geogrid normally needs to run only when the domain/geography settings change.
# Set RUN_GEOGRID=1 to run it; default skips it to save time.
if [[ "${RUN_GEOGRID:-0}" == "1" ]]; then
    log "Running geogrid.exe"
    (cd "$BASE_DIR" && ./geogrid.exe > log.geogrid 2>&1) || {
        tail -n 60 "$BASE_DIR/log.geogrid" >&2 || true
        die "geogrid.exe failed; see $BASE_DIR/log.geogrid"
    }
else
    log "Skipping geogrid.exe (set RUN_GEOGRID=1 to run it)"
fi

log "Linking GRIB files"
(
    cd "$BASE_DIR"
    # Remove old GRIBFILE links so the new cycle cannot mix with old inputs.
    rm -f GRIBFILE.*
    ./link_grib.csh "${gribdir}/"
) || die "Failed to link GRIB input files."

# GFS Vtable is appropriate only when the source data are GFS.
ln -sfn ungrib/Variable_Tables/Vtable.GFS "${BASE_DIR}/Vtable"

log "Running ungrib.exe"
(
    cd "$BASE_DIR"
    ./ungrib.exe > log.ungrib 2>&1
) || {
    tail -n 60 "$BASE_DIR/log.ungrib" >&2 || true
```

```
    die "ungrib.exe failed; see $BASE_DIR/log.ungrib"
}

log "Running metgrid.exe"
(
    cd "$BASE_DIR"
    ./metgrid.exe > log.metgrid 2>&1
) || {
    tail -n 60 "$BASE_DIR/log.metgrid" >&2 || true
    die "metgrid.exe failed; see $BASE_DIR/log.metgrid"
}

# Update WRF run dates, using the actual start cycle and end time.
log "Updating namelist.input"
set_namelist_value "$wrf_namelist" run_hours "$fc"
set_namelist_value "$wrf_namelist" start_year "$sYear"
set_namelist_value "$wrf_namelist" start_month "$((10#$sMonth))"
set_namelist_value "$wrf_namelist" start_day "$((10#$sDay))"
set_namelist_value "$wrf_namelist" start_hour "$((10#$sHour))"
set_namelist_value "$wrf_namelist" end_year "$eYear"
set_namelist_value "$wrf_namelist" end_month "$((10#$eMonth))"
set_namelist_value "$wrf_namelist" end_day "$((10#$eDay))"
set_namelist_value "$wrf_namelist" end_hour "$((10#$eHour))"

# Run real.exe and wrf.exe from the directory containing WRF executables.
# This assumes WPS and WRF executables/namelists share BASE_DIR, as in the
# original script. If your WRF run directory is separate, set WRF_RUN_DIR.
require_executable "${WRF_RUN_DIR}/real.exe"
require_executable "${WRF_RUN_DIR}/wrf.exe"

log "Running real.exe with ${REAL_NP} MPI rank(s)"
(
    cd "$WRF_RUN_DIR"
    mpirun -np "$REAL_NP" ./real.exe > log.real 2>&1
) || {
    tail -n 80 "${WRF_RUN_DIR}/log.real" >&2 || true
    die "real.exe failed; see ${WRF_RUN_DIR}/log.real"
}

log "Running wrf.exe with ${WRF_NP} MPI rank(s)"
(
    cd "$WRF_RUN_DIR"
    mpirun -np "$WRF_NP" ./wrf.exe > log.wrf 2>&1
) || {
    tail -n 80 "${WRF_RUN_DIR}/log.wrf" >&2 || true
```

```
    die "wrf.exe failed; see ${WRF_RUN_DIR}/log.wrf"
}

# Move output files and replace colon characters in timestamps with underscores.
shopt -s nullglob
outputs=( "${WRF_RUN_DIR}"/wrfout_* )
if (( ${#outputs[@]} )); then
    for file in "${outputs[@]}"; do
        [[ -f "$file" ]] || continue
        name="$(basename "$file")"
        safe_name="${name//:/_}"
        safe_name="${safe_name//-/_}"
        mv -- "$file" "${wrfdir}/${safe_name}"
    done
else
    log "WARNING: No wrfout_* files found in ${WRF_RUN_DIR}"
fi

# Clean temporary metgrid links/files from the WRF run directory only.
find "$WRF_RUN_DIR" -maxdepth 1 -type l -name 'met_em*' -delete
find "$WRF_RUN_DIR" -maxdepth 1 -type f -name 'met_em*' -delete
run_end="$(date -u +%s)"
elapsed=$(( run_end - run_start ))
printf '\nWRF run completed.\n'
printf 'Output directory: %s\n' "$wrfdir"
printf 'Elapsed time: %d hour(s), %d minute(s), %d second(s)\n' \
    "$((elapsed / 3600))" "$(((elapsed % 3600) / 60))" "$((elapsed % 60))"
```

Save the script as run_wrf.sh into the $WRF_HOME/scripts directory and execute it

```
chmod +x run_wrf.sh

./run_wrf.sh -auto # In the script, the forecast hours are set to 72 (3 days). When we need to extend, we have to change fc="72" or use the second option below.

# ./run_wrf.sh YYYYMMDD CYCLE FORECAST_HOURS
 ./run_wrf.sh 20261016 00 120
```

Arguments are the initialization date (YYYYMMDD), UTC cycle (00, 06, 12, or 18), and forecast length in hours (1–72).

Run with 2 MPI processes for the real program and 8 MPI processes for the WRF model. The script run_wrf.sh should be in the same directory as this script.

```
REAL_NP=2 WRF_NP=8 ./run_wrf.sh 20261016 00 72
```

Run geogrid

```
RUN_GEOGRID=1 ./run_wrf.sh 20261016 00 72
```

By default, geogrid.exe is skipped because the geographic domain normally needs to be generated only when the domain configuration changes.

Important checks before running

- Confirm that `$WRF_HOME/ethiowrf/` contains the WPS executables, namelist.wps, and the GRIB input directory GFS_InputYYYYMMDDCC/.
- Confirm that **namelist.input,** `real.exe`, and `wrf.exe` are in the directory configured by $WRF_HOME/ethiowrf/.
- Ensure the **`Vtable.GFS`** link is appropriate for your input data. Other meteorological data sources may require a different **Vtable**.
- The script creates timestamped backups of both namelists before editing them.
- Verify that your GRIB files cover the entire forecast period and that your WRF domain and MPI configuration are correct.
- Important: This is a corrected starting point, not a guarantee that a forecast will run successfully. Check **log.ungrib, log.metgrid, log.real,** and **log.wrf** if any stage fails.

Chapter four

4.WRF Post-Processing Using Python

Python is a powerful, flexible, and widely used tool for processing, analyzing, visualizing, and evaluating Weather Research and Forecasting (WRF) model output. It enables meteorologists, climatologists, hydrologists, researchers, and numerical weather prediction (NWP) practitioners to transform raw WRF output into meaningful meteorological products, forecast maps, time series, statistical summaries, and scientific analyses.

4.1 Major applications of Python in WRF post-processing

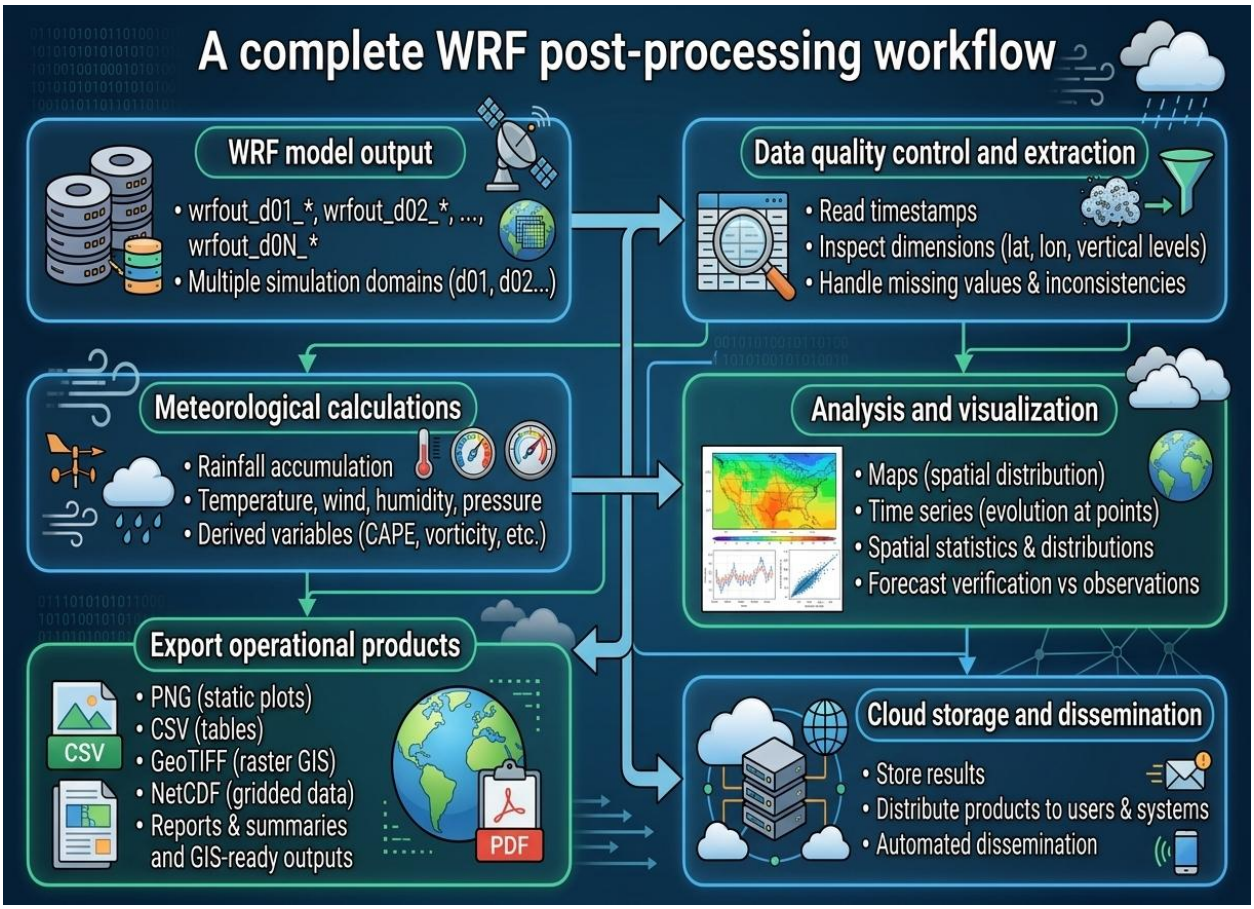
1. Meteorological variable extraction: Extract temperature, precipitation, wind speed and direction, relative humidity, pressure, geopotential height, vertical velocity, and other variables from wrfout NetCDF files.

2. Weather map generation: Produce spatial maps of rainfall, temperature, wind, humidity, pressure, and atmospheric circulation using Matplotlib, Cartopy, and the WRF-Python package.

3. Time-series and temporal analysis: Calculate hourly, daily, monthly, and seasonal statistics, examine forecast evolution, identify rainfall events, and analyze changes in meteorological variables.

4. Wind and atmospheric dynamics: Calculate wind speed and direction, plot wind vectors and streamlines, examine vertical profiles, and analyze atmospheric circulation and storm development.

5. Forecast verification: Compare WRF simulations with station observations, satellite precipitation, reanalysis, or other reference datasets using bias, RMSE, correlation, equitable threat score, and other suitable verification metrics.



4.1.1 Recommended Python libraries

Library	Main purpose
`wrf-python`	Extract and diagnose WRF variables, interpolate fields, calculate meteorological diagnostics
`netCDF4`	Read and inspect WRF NetCDF output
`xarray`	Work with multidimensional and time-dependent data
`numpy`	Numerical calculations and array operations
`pandas`	Time series, station data, and tabular summaries
`matplotlib`	Scientific plots and time-series charts
`cartopy`	Geographic projections, coastlines, borders, and map features
`scipy`	Interpolation, filtering, and statistical analysis
`rasterio`	Export and process geospatial raster products
`geopandas`	Work with administrative boundaries and vector GIS data

4.2 Install the main packages

Create a Bash script named create_wrfpost_env.sh under **/$WRF_HOME/scripts/** to create and activate your Python environment for WRF post-processing.

```
#!/usr/bin/env bash
# ============================================================
# WRF Post-Processing Python Environment
# Location: $WRF_HOME/scripts/
# Environment: wrfpost
# ============================================================

set -Eeuo pipefail

ENV_NAME="wrfpost"
PYTHON_VERSION="3.11"

echo "============================================================"
echo " Creating WRF Post-Processing Conda Environment"
echo "============================================================"

# Check whether Conda is available
if ! command -v conda >/dev/null 2>&1; then
    echo "ERROR: Conda was not found in PATH."
    echo "Initialize or install Miniconda/Anaconda, then retry."
    exit 1
fi

# Initialize Conda for this Bash process
CONDA_BASE="$(conda info --base)"
# shellcheck disable=SC1091
source "${CONDA_BASE}/etc/profile.d/conda.sh"

# Create the environment if it does not already exist
if conda env list | awk '{print $1}' | grep -Fxq "${ENV_NAME}"; then
    echo "Environment '${ENV_NAME}' already exists."
    echo "Updating required packages..."
    conda install -n "${ENV_NAME}" -c conda-forge \
        "python=${PYTHON_VERSION}" \
        wrf-python \
        netcdf4 \
        xarray \
        numpy \
        pandas \
        matplotlib \
        cartopy \
```

```
        scipy \
        geopandas \
        rasterio \
        -y
else
    echo "Creating environment '${ENV_NAME}'..."
    conda create -n "${ENV_NAME}" -c conda-forge \
        "python=${PYTHON_VERSION}" \
        wrf-python \
        netcdf4 \
        xarray \
        numpy \
        pandas \
        matplotlib \
        cartopy \
        scipy \
        geopandas \
        rasterio \
        -y
fi

echo
echo "Activating environment '${ENV_NAME}'..."
conda activate "${ENV_NAME}"

echo
echo "Checking installed packages..."
python --version

python - <<'PY'
import importlib

packages = [
    "wrf",
    "netCDF4",
    "xarray",
    "numpy",
    "pandas",
    "matplotlib",
    "cartopy",
    "scipy",
    "geopandas",
    "rasterio",
]
```

```
failed = []
for package in packages:
    try:
        module = importlib.import_module(package)
        print(f"OK: {package} {getattr(module, '__version__', '')}")
    except Exception as exc:
        failed.append((package, str(exc)))
        print(f"FAILED: {package}: {exc}")

if failed:
    raise SystemExit("One or more package imports failed.")
PY

echo
echo "============================================================"
echo " WRF Post-Processing Environment Ready"
echo " Environment: ${ENV_NAME}"
echo " Script directory: $WRF_HOME/scripts"
echo "============================================================"
echo
echo "To activate it in a future terminal, run:"
echo "  conda activate ${ENV_NAME}"
```

Save it as `create_wrfpost_env.sh` and execute it, `chmod +x create_wrfpost_env.sh`

Run the installer

```
cd $WRF_HOME/scripts
./create_wrfpost_env.sh
```

Once installation completes, activate the environment whenever you need it:

```
conda activate wrfpost
```

Note: The script creates the environment if it is missing, or updates it if it already exists. It checks whether the requested Python packages can be imported. This prepares the Python environment but does not itself run WRF post-processing

Appendix A. Useful commands

```
# Help
bash install_wrf_4.6.0.sh --help

# ARW installation
bash install_wrf_4.6.0.sh -arw

# All requested variants
bash install_wrf_4.6.0.sh -all

# Continue library setup where marker files exist
bash install_wrf_4.6.0.sh -arw --continue

# Alternate installation root
bash install_wrf_4.6.0.sh -arw --homedir "$HOME/WRF_4.6.0"

# Activate ARW environment
source "$HOME/Build_WRF/etc/modulefiles/WRFARW.sh"

# List key executables
find "$HOME/Build_WRF/WRF-4.6.0-ARW" "$HOME/Build_WRF/WPS-4.6.0-ARW" \
  -maxdepth 3 -type f -executable | sort

# Check log errors
grep -iE 'error|fatal|undefined reference|not found' \
  "$HOME/Build_WRF/logs"/*.log | tail -n 100
```

Appendix B. References and assumptions

Primary source: user-supplied Bash script `install_wrf_4.6.0.sh` (552 lines), which names WRF/WPS 4.6.0 and the library versions and URLs reproduced in this guide.

Useful official documentation to consult for version-specific configuration:

- WRF model repository and release notes: https://github.com/wrf-model/WRF
- WPS repository: https://github.com/wrf-model/WPS
- WRF online tutorial and build guidance: https://www2.mmm.ucar.edu/wrf/OnLineTutorial/
- WRF-Hydro public model repository: https://github.com/NCAR/wrf_hydro_nwm_public
- Unidata NetCDF documentation: https://docs.unidata.ucar.edu/netcdf-c/current/

The guide assumes a Linux/Bash user account with outbound network access and GNU compilers. Commands are examples and may require adaptation to a cluster, scheduler, operating system, MPI policy, input-data source, or local filesystem. No successful installation or forecast is implied by the existence of this guide.